

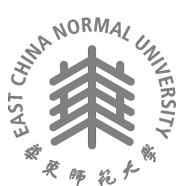
2024 届硕士专业学位研究生学位论文

分类号: _____

学校代码: _____ 10269

密 级: _____

学 号: _____ 71215902050



華東師範大學

East China Normal University

硕士专业学位论文

Master's Degree Thesis (Professional)

论文题目: 面向终端安全的恶意代码
特征检测技术研究

院	系:	软件工程学院
专业学位类别:		电子 信 息
专业学位领域:		软 件 工 程
学位 申 请 人:		尤 乾 玉
指 导 教 师:		刘 虹 研究员

2024 年 4 月 25 日

Thesis for Master's Degree (Professional) in 2024

University Code: 10269

Student ID: 71215902050

EAST CHINA NORMAL UNIVERSITY

Title:
Research on Malicious Code Feature Detection
Technology for Endpoint Security

Department:	Software Engineering Institute
Category:	Electronic Information
Field:	Software Engineering
Candidate:	You Qian Yu
Supervisor:	Liu Hong (Researcher)

April, 2024

尤乾玉硕士学位论文答辩委员会成员名单

姓名	职称	单位	备注
李成举	教授	华东师范大学	主席
曾鹏	教授	华东师范大学	
洪璇	副教授	上海师范大学	

摘 要

攻与防之间始终是一场持续的对抗，而恶意代码的攻击手段正变得日益多样化，这使得终端安全的防护面临着更大的挑战。近年来，随着直接系统调用、无文件攻击、内存加密等新型终端攻击技术的出现，传统的规则检测方法难以有效地应对采用免杀技术的恶意代码攻击，需要提出新算法来弥补现有规则检测的不足。同时，面对特征提取能力有限和样本量不足的问题，检测未知变种的恶意代码变得尤为困难，也需要提出准确率更高、鲁棒性更强的机器学习检测模型。

基于以上问题，本文的主要研究工作如下：

(1) 针对恶意代码利用免杀手段绕过检测的问题，本文提出了基于行为特征的恶意代码检测算法。该算法融合多个检测角度，在动态加载阶段，从堆栈调用和指令地址的角度识别出直接或间接的系统调用等行为；同时，在内存运行阶段，从睡眠追踪和内存属性的角度对内存中的恶意代码进行检测，以判断内存是否受到攻击。实验表明，该检测算法能有效防御采用新型免杀技术的恶意代码攻击。

(2) 针对未知恶意代码变体的准确识别问题，本文提出了基于 Stacking 集成学习的恶意代码检测模型。该模型结合了 XGBoost 和改进的 TextCNN 作为基模型，用于训练恶意代码的 API 调用序列并生成元特征。随后，通过 XGBoost 元模型对元特征进行进一步的训练。实验表明，该检测模型能有效提升恶意代码检测的准确率。

(3) 针对新型免杀技术和未知恶意代码变体带来的风险，本文设计并实现了终端恶意代码特征检测系统。该系统通过检测算法对恶意代码中的免杀行为特征进行规则检测，并运用检测模型对未知恶意代码变种进行分类检测。测试表明，该检测系统在应对采用免杀技术的未知变种恶意代码方面展现出了良好的检测效果。

本文从多个维度探索了终端安全中恶意代码的特征检测技术，不仅为传统规则检测提供了有力的补充，也为机器学习模型检测开拓了新的思路。

关键词： 终端安全、恶意代码检测、免杀技术、机器学习

ABSTRACT

The struggle between offense and defense is always ongoing, with the methods of malicious code attacks becoming increasingly diverse, posing greater challenges to endpoint security protection. In recent years, with the emergence of new endpoint attack techniques such as direct system calls, fileless attacks, and memory encryption, traditional rule detection methods have struggled to effectively counteract malicious code attacks that utilize evasion techniques. This necessitates the development of new algorithms to supplement the deficiencies of existing rule detection. Moreover, faced with the limitations in feature extraction capabilities and insufficient sample volumes, detecting unknown variants of malicious code has become especially challenging. This also calls for the proposal of machine learning detection model with higher accuracy and stronger robustness.

Based on the above issues, the main research works of this thesis are as follows:

(1) To address the issue of malicious code evading detection using evasion techniques, this thesis proposes a detection algorithm based on behavioral characteristics. The algorithm integrates multiple detection perspectives. During the dynamic loading stage, it identifies direct or indirect system calls and other behaviors from the perspectives of stack calls and instruction addresses. Additionally, during the memory execution stage, it examines malicious code in memory from the perspectives of sleep tracking and memory attributes to determine if the memory has been compromised. Experiments demonstrate that this detection algorithm can effectively defend against attacks using novel evasion techniques.

(2) To accurately identify unknown variants of malicious code, this thesis introduces a malicious code detection model based on Stacking ensemble learning. This model integrates XGBoost and an enhanced TextCNN as base models to train on API call sequences of malicious code and generate meta-features. Subsequently, these meta-features are further trained using an XGBoost meta-model. Experiments show that this detection model

significantly enhances the accuracy of malicious code detection.

(3) In response to the risks posed by novel evasion techniques and unknown variants of malicious code, this thesis designs and implements an endpoint malicious code feature detection system. The system utilizes detection algorithms to perform rule-based detection of evasion behavior features in malicious code and applies the detection model to classify unknown variants of malicious code. Tests demonstrate that this detection system exhibits good performance in detecting unknown variants of malicious code that employ evasion techniques.

This thesis explores malicious code feature detection technology in endpoint security from multiple dimensions, not only offering a significant enhancement to traditional rule-based detection but also pioneering new approaches for machine learning model detection.

Keywords: *Endpoint security, Malicious code detection, Evasion techniques, Machine learning*

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究与现状.....	3
1.2.1 传统恶意代码检测技术.....	3
1.2.2 基于机器学习的检测技术.....	4
1.3 论文的主要工作.....	6
1.4 论文的组织结构.....	7
第二章 相关知识和技术研究	9
2.1 恶意代码介绍.....	9
2.1.1 恶意代码定义及分类.....	9
2.1.2 恶意代码检测技术.....	10
2.2 恶意代码免杀技术.....	12
2.3 机器学习模型.....	14
2.3.1 决策树算法.....	15
2.3.2 卷积神经网络.....	16
2.4 集成学习策略.....	18
2.5 本章小结.....	19
第三章 基于行为特征的恶意代码检测算法	20
3.1 检测算法的基本框架.....	20
3.2 动态加载阶段的特征检测	21
3.2.1 动态加载阶段的行为特征分析.....	21
3.2.2 动态加载阶段的特征检测算法设计.....	21
3.3 内存运行阶段的特征检测	27
3.3.1 内存运行阶段的行为特征分析.....	27
3.3.2 内存运行阶段的特征检测算法设计.....	27
3.4 实验与结果分析.....	31
3.4.1 实验环境.....	31
3.4.2 结果分析.....	33
3.5 本章小结.....	34
第四章 基于 Stacking 集成学习的恶意代码检测模型	36
4.1 检测模型的基本框架.....	36
4.2 XGBoost 与改进 Text CNN 的基模型训练.....	37
4.2.1 XGBoost 模型训练.....	37
4.2.2 改进 Text CNN 模型训练	41

4.3 采用 Stacking 集成学习的元模型训练	45
4.4 实验与结果分析.....	47
4.4.1 实验环境与数据	47
4.4.2 评价方法及指标.....	48
4.4.3 结果分析.....	49
4.5 本章小结.....	52
第五章 终端恶意代码特征检测系统设计与实现	53
5.1 需求分析.....	53
5.2 系统设计与实现.....	54
5.3 系统测试.....	55
5.3.1 测试环境.....	55
5.3.2 针对现有免杀技术的功能测试.....	56
5.3.3 恶意代码特征检测系统的性能测试.....	57
5.4 测试结果分析.....	58
5.5 本章小结.....	59
第六章 全文总结与展望	60
6.1 研究工作总结.....	60
6.2 研究展望.....	61
参考文献.....	62

图目录

图 1.1	2023 年恶意代码类型统计图	1
图 1.2	论文组织结构图	7
图 2.1	基于传统规则的恶意代码检测流程图	11
图 2.2	基于机器学习的恶意代码检测流程图	12
图 2.3	用户态与内核态示意图	13
图 2.4	决策树结构图	15
图 2.5	CNN 结构图	17
图 2.6	Stacking 架构图	18
图 3.1	检测算法的基本框架	20
图 3.2	正常程序系统回调示意图	21
图 3.3	API 合法调用流程图	22
图 3.4	间接系统调用示意图	23
图 3.5	硬件断点的间接系统调用示意图	25
图 3.6	指向恶意代码的返回地址示意图	28
图 3.7	内存可执行属性更改示意图	30
图 4.1	检测模型的基本框架	36
图 4.2	XGBoost 模型训练流程图	38
图 4.3	生成 2-Gram API 序列示意图	39
图 4.4	改进 TextCNN 模型结构图	41
图 4.5	改进 TextCNN 模型训练流程图	43
图 4.6	采用 Stacking 集成学习进行模型融合的流程图	46
图 4.7	改进 TextCNN 对检测结果的影响	50
图 4.8	模型融合对检测结果的影响	51

图 5.1 系统框架图	54
-------------------	----

表目录

表 3.1	实验环境配置	32
表 3.2	直接或间接系统调用开源项目介绍	32
表 3.3	内存规避开源项目介绍	33
表 3.4	动态加载阶段检测算法测试结果	33
表 3.5	内存运行阶段检测算法测试结果	34
表 4.1	XGBoost 模型参数及其值	40
表 4.2	改进 TextCNN 模型参数及其值	45
表 4.3	实验环境参数	47
表 4.4	恶意代码样本统计	48
表 4.5	改进 TextCNN 对检测结果的影响	49
表 4.6	模型融合对检测结果的影响	51
表 5.1	实验环境配置	55
表 5.2	组合免杀技术的样本方案	56
表 5.3	针对现有免杀技术的功能测试	57
表 5.4	钓鱼木马的样本信息	57
表 5.5	终端恶意代码特征检测系统的检查结果	58

第一章 绪 论

1.1 研究背景及意义

随着互联网和计算机技术的日益普及，终端安全逐渐成为了国内外研究的焦点。在终端安全领域，恶意代码，包括病毒、蠕虫、木马等多种形态，一直是威胁终端安全和用户隐私的主要因素。这些恶意程序能够盗取敏感信息、破坏数据的完整性、劫持系统资源，并在用户不知情的情况下为攻击者提供持续的控制权限。随着信息技术的快速发展和互联网的广泛应用，新型恶意代码层出不穷，其潜藏技术和攻击手段日益狡猾复杂。

根据瑞星公司发布的《2023 中国网络安全报告》^[1]，报告期内，瑞星“星核”平台共截获病毒样本总量 8456 万个，病毒感染次数 9052 万次，病毒总体数量比 2022 年同期增长了 14.98%。报告期内，新增木马病毒 5312 万个，为第一大种类病毒，占到总体数量的 62.81%；排名第二的为蠕虫病毒，数量为 1577 万个，占总体数量的 18.65%；后门、灰色软件、感染型病毒分别占到总体数量的 10.15%、6.36% 和 1.18%。图 1.1 展示了 2023 年恶意代码各类型在总体中的占比统计，可见恶意代码对终端构成了严重的安全威胁。

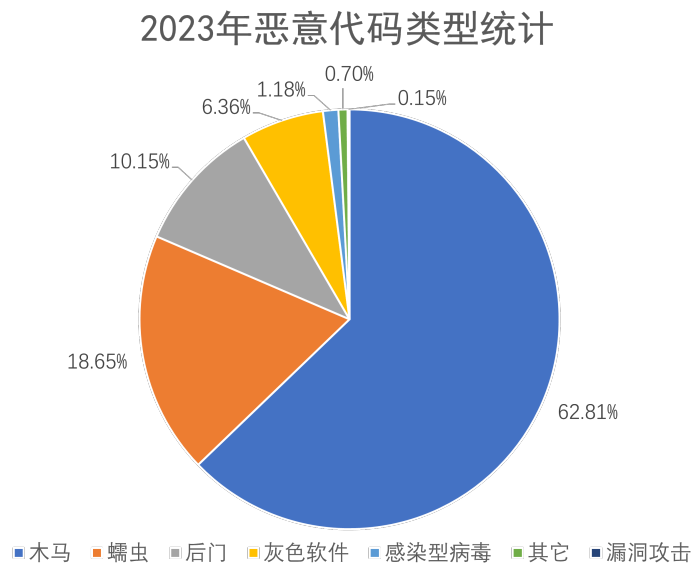


图 1.1 2023 年恶意代码类型统计图

攻击终端的方式正变得越来越复杂，攻击者会利用网络钓鱼、社会工程学原理和漏洞利用等多种技术来攻击目标。同时，双重勒索成为了常态化攻击模式，攻击者不仅仅专注于对受害者文件进行加密，而是更加倾向于通过泄露敏感数据作为敲诈勒索的筹码，给政府或企业受害者带来了更大的压力。即便受害者通过文件恢复的方式解密数据，敏感数据泄露带来的重大风险和代价也是难以承受的。传统的恶意代码检测技术多基于已知特征和行为模式，依赖于病毒库进行匹配，但面对未知或变种恶意代码时，这种方法容易受限，漏洞率高。为了解决这一问题，研究者开始探索基于机器学习的恶意代码检测技术，目的是通过学习大量样本的特征模式，提高对新型恶意代码的识别能力。

目前，终端检测通常结合多种检测引擎，包括静态分析引擎、行为检测引擎和机器学习引擎。静态检测引擎主要通过分析程序代码或二进制执行文件的静态特征来识别恶意代码，如签名匹配、哈希值对比、字符串搜索等，这种方法可以快速识别已知的恶意代码，但对于加密或混淆过的代码检测效果有限。行为检测引擎则是通过监控程序的运行行为来判断其是否具有恶意性。这种方法不依赖于恶意代码的静态特征，能够有效识别出利用加密、混淆或未知漏洞的新型恶意代码。行为检测通常需要运行在一个受控环境中，如沙箱，以隔离潜在的恶意行为，避免对实际系统造成损害。机器学习引擎通过分析大量的数据样本来自动识别出恶意代码的模式和特征。这种方法可以通过学习不断优化和调整检测模型，以适应新出现的恶意代码和攻击技术。机器学习引擎特别适用于处理复杂的数据分析任务，能够有效提高恶意代码检测的准确率和速度。

综上所述，静态分析技术在面对不断涌现的隐蔽恶意代码时显得力不从心。相比之下，行为检测和机器学习技术以其高效的检测能力和广泛的适应性，成为了杀毒软件（Antivirus, AV）与终端检测与响应^{[2][3]}（Endpoint Detection and Response, EDR）的重要支柱。终端安全本质上是一场攻防对抗，攻击技术的进步不断推动防御技术的发展。本文围绕面向终端安全的恶意代码特征检测技术，旨在设计和实现一种能够有效检测未知恶意代码的新型检测方法，提高检测的准确率和速度，减少误报和漏报，为终端安全防护提供坚实的技术支撑。主要探讨了针对 Windows 终端的行为检测和机器学习在恶意代码检测中的应用。结合行为分析和机器学习

技术不仅可以提高恶意代码的检测准确率，还能有效降低误报率，为终端安全防护提供了一种更为高效和动态的解决方案。

1.2 国内外研究与现状

由于恶意代码对终端安全的威胁从未停止，国内外一直都有对恶意代码检测的研究。目前国内外对恶意代码检测的研究分为两个方面：传统恶意代码检测技术；基于机器学习的检测技术。

1.2.1 传统恶意代码检测技术

传统的恶意代码检测通常结合使用特征码匹配和语义分析的策略。特征码匹配，一种广泛采用的早期技术，通过比较代码与一个包含已知恶意代码特征的数据库进行匹配来识别恶意样本。这些数据库收录了如文件名、字节模式等关键信息。与之相对的，语义分析技术则旨在识别由代码混淆^[4]引起的恶意代码变种，它基于代码的语义内容而非仅仅是语法形式，因混淆虽改变代码外观但不改变其本质功能。这种方法深入分析反汇编代码，解析其指令意图，并构建如函数调用图、控制流图等分析工具，进而洞察程序实际行为。

Sung^[5]等人提出了一种基于系统调用的静态恶意代码检测方法，该方法通过反汇编恶意代码以提取其中的文本信息，进而获得系统调用序列。然后，它通过比较系统调用序列的相似度来检测恶意代码的变种。Karnik^[6]等人提出了一种基于统计技术的恶意代码特征检测方法，该方法通过余弦相似度来比较恶意代码，对于识别已知的混淆恶意代码显示出了良好的效果。王晓洁^[7]等人提出了一种基于语义匹配的检测算法，该算法能够准确检测出经过代码混淆处理的恶意代码。通过应用数据流分析技术，并以变量定义使用链作为检测单元，对每个模板及程序节点进行了详细分析。最终，通过部分实验，展示了该原型系统的有效检测效果。王蕊^[8]等人提出了一种结合污点传播和语义分析的恶意代码检测方法。该方法通过分析恶意代码的行为特征及其相互依赖，进而识别和抵抗混淆技术。构建的原型系统在多个样本测试中展现了出色的检测准确性和抗干扰性能。Cesare^[9]等人提出了一种基于控制流图相似性搜索的恶意代码变种检测方法。该方法采用距离

度和最小匹配距离算法来识别恶意代码变种，通过分析恶意代码的控制流图集合并运用字符串编辑距离及线性求和分配技术，实现对图集合之间相似度的量化。

尽管特征码匹配和语义分析在恶意代码检测中发挥了关键作用，但这些方法在识别未知恶意代码、对抗代码混淆技术以及适应恶意代码的快速演变方面存在明显限制。首先，这些技术往往缺乏足够的泛化能力来有效检测未知样本；其次，混淆技术所产生的变种程序易于干扰检测过程，使得特征码库需频繁更新，从而大量占用资源并降低检测效率。总而言之，这些方法在对抗当前的恶意代码方面仍存在不足。

1.2.2 基于机器学习的检测技术

利用机器学习检测恶意代码是终端安全领域中研究的热点，它通过应用数据科学和人工智能的方法^[10]来提升对恶意代码及其他安全威胁的识别效率与准确性。这种技术依赖于从大量数据中学习恶意与正常代码的区分特征，包括文件的静态属性和行为特征等，以自动化的方式识别新的安全威胁。通过 K-近邻 (KNN)^[11]、朴素贝叶斯 (NB)^[12]、决策树 (DT)^[13] 和支持向量机 (SVM)^[14] 等机器学习模型在训练阶段学习已知样本的特征，进而能够对未知样本进行有效的检测和分类。此外，模型能够通过持续学习新的恶意代码样本不断优化，提高检测的准确率。

孙博文^[15]等人提出了一种基于多特征融合的恶意软件分类方法。通过分析 Virusshare 提供的恶意样本与卡巴斯基引擎的扫描结果，该方法从字节视图、汇编视图和 PE 视图三个维度提取特征，并采用了特征融合与算法集成的策略。通过测试和对比 SVM、决策树、随机森林等多种机器学习算法，经过精细的优化和调整，该方法成功实现了 93.56% 的高准确率。Chen^[16]等人提出了一种基于数据挖掘的恶意代码检测方法，旨在克服基于标签的检测系统的局限。该方法首先提取软件的一系列静态特征，然后在预处理阶段应用 PCA 算法进行特征降维。在决策过程中，采用 XGBoost 算法来实现高度准确的恶意代码识别。Louk^[17]等人则采取了一种基于集成学习的方法来检测恶意代码，通过结合 GBM 和 LightGBM 等多种树模型。这项研究发现，所有选取的基于树的算法在恶意代码的检测任务上都展

现出了显著的效果，且在三个不同的数据集上都取得了卓越的检测性能。Akshit^[18]等人提出了一种基于多种机器学习模型的方法来区分恶意样本和良性样本。这些模型包括决策树、随机森林、梯度提升、XGBoost 算法和 AdaBoost 算法（自适应提升）。Tran^[19]等人提出了一种基于将恶意代码二进制文件转换为图像纹理的方法，使用 KNN、支持向量机和卷积神经网络等机器学习分类器进行恶意代码的检测和分类。该方法通过在模型训练之前缩小恶意代码图像的宽度，以提高分类性能并减少模型训练时间。

深度学习^[20]属于机器学习的子领域。不同于传统机器学习算法需要手工特征工程，耗时且影响模型性能，深度学习算法可以自动提取特征，有效地处理大规模数据，从而实现对恶意代码的高准确率检测。这些技术通常包括卷积神经网络 (CNN)^{[21][22]}、循环神经网络 (RNN)^[23]、长短期记忆网络 (LSTM)^[24]和自动编码器^[25]等深度神经网络结构，它们能够学习代码或程序行为中的深层特征。通过训练，深度学习模型能够识别新出现的恶意代码变种，从而为终端安全提供更加强大的防护。

Tobiyama^[26]等人提出了一种恶意软件进程检测方法，基于被感染终端的进程行为，通过逐步应用递归神经网络和卷积神经网络进行深度学习分类。此方法在最佳情况下达到了 0.96 的 AUC 评分。Catak^[27]等人提出了一种基于深度学习的新方法，通过构建包含多层卷积和循环网络层的神经网络，以提高恶意软件的自动检测和分类准确率。该模型通过有效提取恶意软件的系统调用序列特征，优化了恶意软件分类过程。王天歌^[28]提出一种融合 Word2vec 和注意力机制的卷积神经网络恶意代码检测模型。这个模型通过将 API 调用转换为向量，并使用不同尺寸的卷积核及注意力层优化网络结构，从而提高了恶意代码检测的准确性。Xu^[29]等人提出了一种基于程序控制流图和数据流图的方法。这种方法涉及将程序控制流图和数据流图转换成矩阵，然后映射为图像，并使用传统的 CNN 模型进行微调和分类。该方法通过将控制流图和数据流图及其可能的组合作为应用程序的描述特征，进而将这些图编码成矩阵，并利用卷积神经网络来训练分类模型。张孟哲^[30]提出了一种基于 API 序列的 CBLA 模型，通过 API2Vec 技术和 BiLSTM 网络捕获 API 调用的关系和顺序，结合注意力机制聚焦关键恶意序列，有效提升恶意代码检

测性能。

使用深度学习模型进行恶意代码检测需要大量用于训练的标注数据，而且这些模型往往在解释性方面表现不佳，这些都是当前研究需要解决的挑战。总的来说，尽管基于机器学习和深度学习的技术在自动化和准确性方面展现出明显的优势，但它们仍然面临一些问题，例如：训练高准确率模型所需的高质量、广泛覆盖的样本，模型复杂性带来的计算资源大量需求，以及恶意攻击者采用的对抗性策略^[31]可能导致检测回避。虽然存在这些挑战，但随着技术的持续发展，基于机器学习的检测技术仍将在终端安全防护方面扮演更加重要的角色。

1.3 论文的主要工作

本文对从传统规则检测和机器学习检测展开研究，提出了通过行为特征检测恶意代码的算法和模型。最后实现能够多维度检测的终端恶意代码特征检测系统。以下本文主要的工作内容：

(1) 本文研究了基于行为特征的恶意代码检测算法，旨在准确发现动态加载及内存运行阶段的恶意行为。该算法通过定义恶意行为规则，在动态加载阶段准确识别直接或间接的系统调用等恶意行为，并在内存运行阶段利用睡眠追踪和内存属性技术来识别内存中驻留的恶意代码。这种方法不仅加强了终端安全防护，还能有效抵御采用新型免杀技术的恶意代码攻击。

(2) 本文研究了基于 Stacking 集成学习的恶意代码检测模型，旨在提高未知恶意代码变体的识别准确率。该模型通过融合 XGBoost 的强大数据拟合能力和改进 TextCNN 模型的文本数据深层次理解能力，实现了恶意代码的高准确率检测。采用 Stacking 集成学习策略，该模型有效整合了两种模型的优势，不仅准确识别了未知变体恶意代码，而且在提升检测精度方面展现了良好的性能。

(3) 本文研究并实现了终端恶意代码特征检测系统，旨在应对采用新型免杀技术的未知恶意代码变体带来的挑战。该系统融合传统规则检测与机器学习的强项，运用规则检测算法挖掘恶意代码中的逃避技术特征，同时利用机器学习模型对未知恶意代码变种进行准确检测。该系统能够对采用免杀技术的未知变种恶意代码具有较好的检测能力。

1.4 论文的组织结构

本文主要章节研究内容围绕图 1.2 所示的组织结构展开，具体各章的内容安排如下：

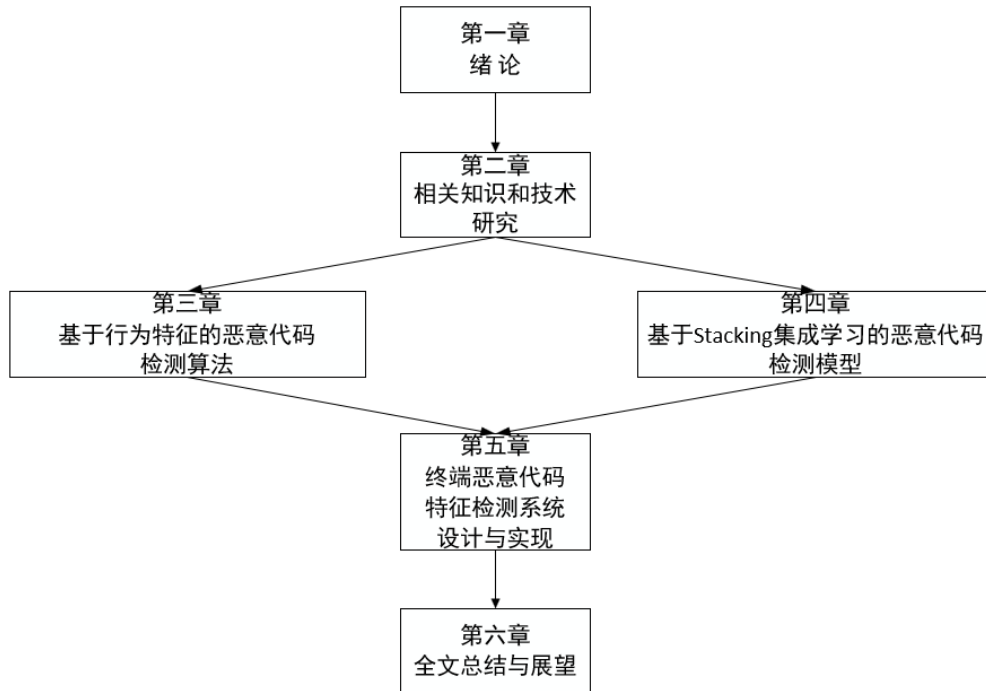


图 1.2 论文组织结构图

第一章绪论。阐述了研究背景及其意义，并对当前恶意代码检测研究进行了调研。明确本文研究的主要内容，总结本文的主要工作。

第二章介绍了恶意代码的定义及其检测技术，并阐述了本文中涉及的恶意代码免杀技术、机器学习模型以及集成学习策略的相关理论基础。

第三章提出了基于行为特征的恶意代码检测算法。本章阐述了检测算法的整体框架和实现流程，包括针对恶意代码动态加载与内存运行阶段的行为特征及相应的检测方法，最后对提出的检测算法进行实验。

第四章提出了基于 Stacking 集成学习的恶意代码检测模型。本章运用 Stacking 集成学习策略将 XGBoost 与改进 TextCNN 模型融合，最后对本章提出的模型进行了实验分析。

第五章提出了终端恶意代码特征检测系统。本章结合传统规则检测和机器学习技术，设计并实现了这一多维度终端恶意代码检测系统，并对系统的整体架构以及各个功能模块进行描述。

第六章对全文的工作内容进行了总结，并对未来恶意代码检测的研究方向进行了展望。

第二章 相关知识和技术研究

本章对论文中的相关理论及技术进行阐述，介绍了恶意代码检测技术、恶意代码免杀技术、机器学习模型以及集成学习策略。

2.1 恶意代码介绍

2.1.1 恶意代码定义及分类

恶意代码是设计用来对计算机系统、网络或设备执行未授权的或恶意活动的软件。它们可以采取多种形式，每种都有其特定的行为和攻击目标。以下是一些常见类型的恶意代码及其介绍：

(1) 勒索病毒：勒索软件通过加密用户的文件和数据，然后要求支付赎金来解锁。这种恶意软件利用用户对数据的需求，通过威胁永久删除或保持加密状态来强制用户支付赎金。

(2) 挖矿程序：恶意挖矿软件悄无声息地使用受害者的计算资源来挖掘加密货币。这种软件往往在后台运行，消耗 CPU 和 GPU 资源，导致设备性能下降。

(3) DDoS 木马：这种恶意软件使攻击者能够通过受感染的设备发起分布式拒绝服务攻击，目的是使目标网站或网络服务不可用，通常用于勒索或其他恶意目的。

(4) 蠕虫病毒：蠕虫是一种自我复制的恶意软件，它不需要与任何程序附着即可传播。蠕虫利用网络连接，在无需用户干预的情况下，自动从一个计算机传播到另一个计算机。

(5) 感染型病毒：这种病毒通过附着在可执行文件或文档上来传播，当这些文件被执行或打开时，病毒就会激活。它们可以破坏文件、盗取数据或创建后门以便于远程访问。

(6) 后门程序：后门程序为攻击者提供了绕过正常认证过程的方式，直接访问受感染的计算机或网络。这使攻击者能够远程控制受感染的系统，窃取数据或部署其他恶意软件。

(7) 木马程序：木马伪装成合法软件或隐藏在合法软件内，诱导用户下载和安装。一旦激活，它们可以执行多种恶意活动，包括窃取信息、下载其他恶意软件或使系统易受其他攻击。

2.1.2 恶意代码检测技术

恶意代码检测技术在维护网络安全和保护计算机系统免受恶意软件攻击方面起着至关重要的作用。随着恶意软件技术的不断进步和变化，恶意代码检测方法也在不断发展，以适应新的挑战。目前，这些检测技术主要可以分为两大类：传统恶意代码检测方法和基于机器学习的检测方法。每种方法都有其独特的优势和局限性，而在实际的安全防护实践中，往往需要将多种方法结合使用以达到最佳的检测效果。

(1) 基于传统规则的恶意代码检测方法

静态检测方法作为恶意软件识别的基石，依托于对程序文件的直接分析，旨在未执行前揭示其潜在的恶意特性。这种方法涵盖了对二进制代码、程序中的字符串、API 调用序列^[32]、以及文件头信息^[33] 等多个方面的审查。它的核心优势在于能够迅速地处理并筛选大量数据，为安全专家和自动化系统提供了一种高效的初步筛查手段。特别是对于大规模的安全监控系统而言，静态检测能够大幅降低对恶意软件响应时间。尽管如此，静态检测面临的挑战也十分明显。随着恶意软件开发者不断采用更加复杂的混淆和加密技术，静态检测的有效性受到了限制。它们很难识别那些表面特征不明显或已经被改变的新型恶意代码，特别是那些设计来规避传统检测方法的恶意软件。

与静态检测不同，动态检测关注的是程序在执行过程中的行为表现。通过监控其对系统资源的调用、网络活动、以及对其他程序的交互，动态检测方法能够更准确地捕捉到恶意软件的实际行为。这包括识别恶意软件试图隐藏其行为、窃取用户数据或破坏系统安全的各种尝试。动态检测最大的优势在于其能够识别那些静态检测无法预见的复杂恶意行为，包括那些新型和高度变异的恶意软件。此外，通过在隔离的环境中执行恶意软件，动态检测可以在不危及主系统安全的前提下，获取深入的恶意软件行为分析。然而，动态检测的实施成本较高，不仅因为需要

模拟或构建安全的测试环境，还因为其分析过程通常比静态检测更为复杂和耗时。此外，存在着误报的风险，因为某些合法软件在特定情境下的行为可能与恶意软件相似。如图 2.1 是基于传统规则的恶意代码检测流程图。

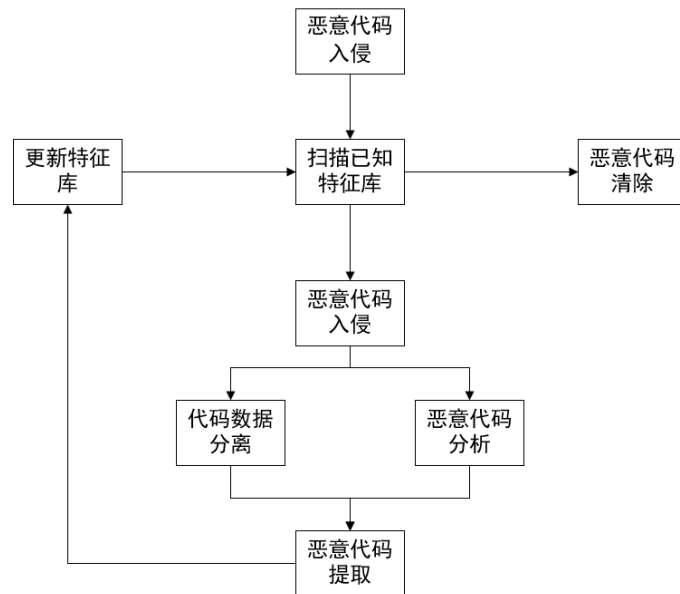


图 2.1 基于传统规则的恶意代码检测流程图

(2) 基于机器学习的恶意代码检测方法

随着人工智能技术的持续进步，基于机器学习的恶意代码检测方法逐渐成为网络安全领域的重要力量。通过深入分析和学习大量已知的恶意软件及其与正常软件的对比样本，这些方法能够构建出能识别恶意软件特定特征和行为模式的高效模型。这类方法利用从数据中自动提取的特征，如 API 调用、内存获取^[34]、注册表篡改等，通过复杂的算法模型如决策树、支持向量机、神经网络等进行学习，使之能够区分恶意行为与正常行为。其核心优势在于模型一旦训练完成，便能持续学习和适应恶意软件特征的变化，这为检测新出现的恶意代码提供了可能。相比传统的基于规则的方法，这种动态学习的能力显著提高了对新型和未知恶意软件的检测能力。此外，机器学习方法在处理和大规模数据集方面表现出色，这对于应对当前日益增长的数据量具有重要意义。这不仅提高了恶意代码检测的速度，也提高了其准确性，减少了误报和漏报的可能性。

尽管基于机器学习的方法具有明显的优势，但在实践中也面临着不少挑战。首

先,模型的性能高度依赖于训练数据的质量和多样性。高质量的、覆盖广泛类型恶意软件的大规模数据集是训练有效模型的前提。然而,获取这样的数据集既困难又耗时,尤其是对于新型恶意软件的样本。其次,模型训练和优化过程通常需要大量的计算资源,包括高性能的处理器和大量的内存,这对于资源有限的组织来说可能是一个挑战。此外,随着模型复杂度的增加,如深度学习模型,这种需求更为明显。最后,虽然基于机器学习的方法在自动化检测方面具较大优势,但其模型的黑盒特性也使得结果难以解释。这在一定程度上限制了安全分析师对检测结果背后逻辑的理解,可能在某些情况下影响恶意软件的进一步分析和响应。如图 2.2 是基于机器学习的恶意代码检测流程图。

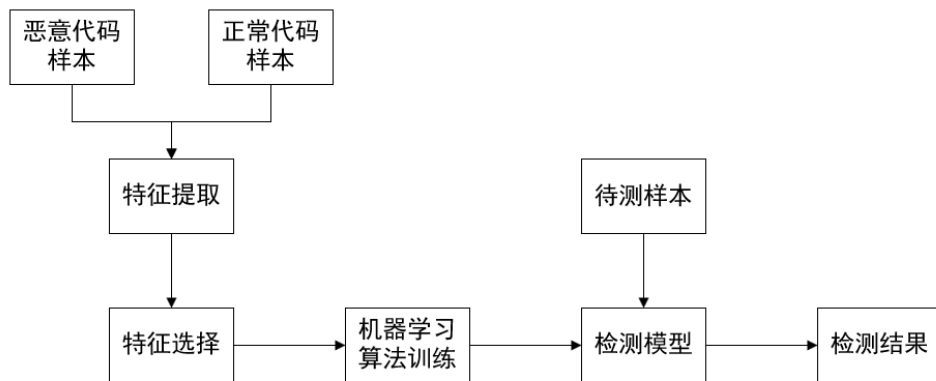


图 2.2 基于机器学习的恶意代码检测流程图

2.2 恶意代码免杀技术

本节主要介绍了恶意代码的免杀技术,包括动态加载和内存运行阶段的技术。同时,还介绍了用于防御免杀技术的 ETW 技术。ETW 被应用于监控系统事件,及时发现恶意行为。以下是详细的介绍。

(1) 随着安全软件,特别是杀毒软件和 EDR,越来越频繁地采用对敏感函数的挂钩(Hook)来监控潜在的恶意行为,攻击者发展出了通过直接或间接系统调用来绕过这类检测的技术。这种绕过技术的核心在于规避杀软对用户态 Windows API 调用的监控。Windows 操作系统支持用户态和内核态两种处理器访问模式。由于操作系统的核心数据结构和内部例程都位于内核态,因此用户态应用程序在需

要执行特权操作时，必须通过系统调用从用户态切换到内核态。如图 2.3 是用户态与内核态的示意图。

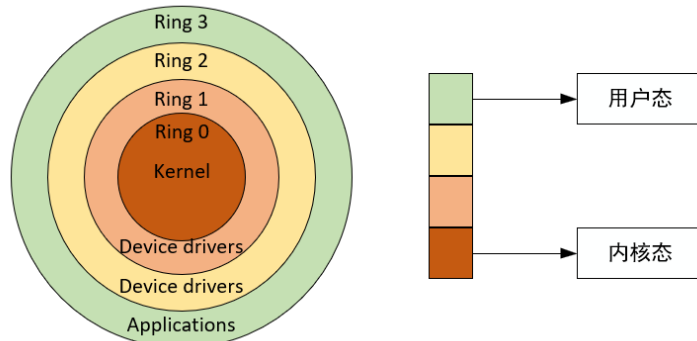


图 2.3 用户态与内核态示意图

EDR 通常会通过挂钩用户态的 Windows API 调用，如 VirtualAlloc（监控内存分配）CreateThread（监控线程创建），来监控进程的动态行为。虽然这种挂钩可以在用户态或内核态完成，但由于用户态挂钩对系统性能的影响较小并且更稳定，不易导致系统崩溃，因此许多 EDR 偏好在用户态进行挂钩。直接或间接系统调用是一种绕过 EDR 用户态挂钩检测的方法。通过获取系统调用号并构造系统调用存根的汇编指令，这种方法允许直接调用内核态 API，从而规避了用户态挂钩的监控。然而，使用这种方法时也可能产生新的检测特征，例如系统调用存根中的硬编码，以及基于栈回溯的检测方法等。因此，不同的系统调用项目也提供了针对这类检测的绕过策略。

（2）内存加密技术是一种高级规避恶意软件检测的方法。它依赖于杀毒软件和 EDR 系统不会持续监控内存，而是定期扫描敏感区域。通过在 Shellcode 执行 Sleep 函数期间对内存区域进行加密，并在休眠结束时解密，恶意代码可以避开安全软件的扫描，从而实现免杀目的。

EDR 可以通过监控关键内核 API 调用来捕获恶意进程尝试分配或修改具有执行权限的内存区域（如将 RW 内存改为 RX）。这是因为 RX 属性通常与 Shellcode 或反射型 DLL 加载相关，EDR 会扫描这些内存请求并在匹配到恶意软件特征时终止进程并发出警报。

利用 Sleep 函数对 Shellcode 进行属性修改和加密,可以在一定程度上规避 EDR 的内存扫描。恶意代码在执行 Sleep 函数时,会暂时将其内存加密并设置为非执行状态,绕过 EDR 检查。休眠结束后,Shellcode 被恢复为可执行状态,等待下一次活动。

通过移除内存的执行属性来避开安全软件的扫描是一种策略,但这会导致恶意代码无法执行。为了解决这一问题,可以利用 Windows 进程在触发异常时的异常处理机制,在需要时恢复内存的执行属性,使恶意代码得以执行。

(3) ETW (Event Tracing for Windows) [35][36]Event Tracing for Windows (ETW) 是由微软开发的一款强大的性能监控和故障诊断工具,专为 Windows 操作系统设计。它为开发者和系统管理员提供了一个高效的机制,用以捕获和跟踪系统级及应用级事件,确保在最小性能开销下进行操作,从而保证即便在高负荷的生产环境中也能安全地进行监控。ETW 的架构包含三个主要元素:提供者 (Providers)、消费者 (Consumers) 和会话 (Sessions)。提供者是事件的源,可能是操作系统的一个组成部分(如内核)或任何外部应用程序;消费者则是订阅并接收事件数据的实体,通常用于数据分析和系统监控;会话控制着数据的收集流程,决定哪些提供者的数据将被捕获及其处理方式。

ETW 作为一个高级的性能监控和诊断工具,被广泛应用于追踪操作系统、应用程序和驱动程序层面的事件,收集关于软件和硬件操作的细节。它支持包括性能分析、故障诊断和安全监控在内的多种应用场景,并能提供 CPU 使用率、磁盘 I/O 和网络活动等详细数据。ETW 的数据可以通过 Windows 内置的工具如事件查看器和性能监视器进行查看,也可以利用各种第三方工具进行深入分析。

2.3 机器学习模型

机器学习和深度学习在推进人工智能技术发展方面发挥了关键作用,使计算机能够通过学习和接纳新知识,不断自我完善,模仿人类的学习过程。通过对大量数据中的模式和规律进行分析和学习,它们可以执行复杂的预测或做出精准的决策。在机器学习的众多模型中,决策树算法和卷积神经网络分别作为传统机器学习和深度学习领域的经典代表,展现了各自独特的优势和广泛的应用前景。

2.3.1 决策树算法

决策树算法采用树形结构，包括根节点、内部节点和叶节点，用于实现分类和回归任务。其核心在于模拟决策过程，每个内部节点对应数据特征的一个测试，每个分支代表测试结果，每个叶节点对应一个预测结果或类别。这种模型因其清晰的逻辑和易于理解的结构而受到众多数据科学家和 AI 研究者的青睐。它不仅能自然地处理分类和回归问题，还能自动发现并利用数据中的重要特征进行有效分割。如图 2.4 是决策树的结构图。

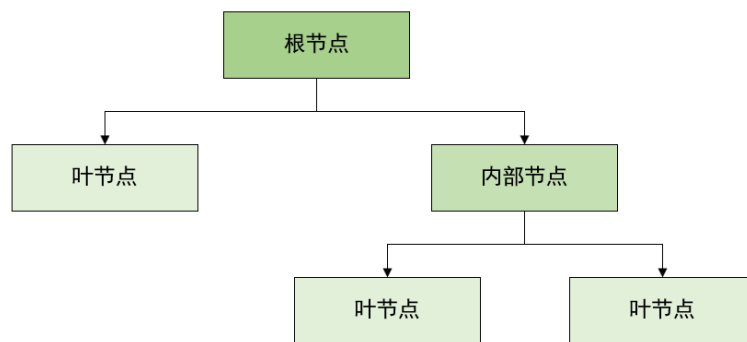


图 2.4 决策树结构图

在决策树算法中，ID3、C4.5 和 CART 是最常见的几种算法。ID3 算法是最早的决策树学习算法之一，它主要通过信息增益来选择特征，构建决策树。C4.5 算法是 ID3 算法的改进版本，它引入了信息增益比来选择特征，解决了 ID3 算法倾向于选择取值多的特征的问题，并且 C4.5 还支持连续特征的处理和树的剪枝操作，以减少过拟合的风险。CART 算法则采用 Gini 指数作为划分标准，生成的是二叉树，它的易于理解和实现，以及在分类效果上的优势，使得 CART 在实际应用中非常受欢迎。通过决策树算法，可以根据数据特征的不同，逐层判断并最终得到分类结果。这些算法的多样性和灵活性使得决策树成为解决分类问题的有力工具。

尽管决策树在处理包含缺失数据和异常值的数据集时显示出相当的稳健性，但其在面对复杂数据集时容易过拟合，并对数据中的微小变化敏感。为了克服这些局限性，引入了集成学习技术，通过整合多个决策树来提升模型的泛化能力和稳定性。决策树集成算法利用多个决策树来提高预测的准确性和模型的鲁棒性。这

种方法背后的基本思想是集成多个学习器的预测结果，通常能够比单一决策树表现得更好。在众多决策树集成算法中，随机森林（RF）^{[37][38]} 和 XGBoost^{[39][40]} 是两个非常流行且有效的模型。

随机森林是一种集成学习技术，它通过构建多棵决策树并将它们的预测结果进行汇总得到最终结果。在随机森林中，每棵树都是独立构建的，并且在训练过程中引入了随机性。具体来说，随机森林在构建每棵树时会随机选择数据集的不同子样本，并且在选择分裂特征时也进行随机选择。这种构建方法不仅增加了模型的多样性，还提高了对新数据的泛化能力。随机森林的优点在于它对于过拟合具有很强的抵抗力，同时对输入特征的缺失数据不敏感，可以很好地处理分类和回归任务。

XGBoost，即 eXtreme Gradient Boosting，是一种基于树的集成学习算法。XGBoost 在传统梯度提升决策树（GBDT）的基础上做了许多优化。它不仅利用了梯度提升的策略，即逐步添加新模型来修正之前模型的错误，还引入了正则化项来控制模型的复杂度，从而防止过拟合。XGBoost 具有高度的灵活性，可以自定义优化目标和评价标准，支持多种语言接口，并且在运算效率和模型性能上都有出色表现。在处理大规模数据时，XGBoost 特别有效，这归功于其对计算资源的优化，包括并行处理和剪枝策略。XGBoost 已经在许多机器学习竞赛中证明了其卓越的性能。

2.3.2 卷积神经网络

卷积神经网络（CNN）^{[41][42]} 由纽约大学的 LeCun 教授首次提出以来，在深度学习领域成为了一个里程碑式的创新，尤其在图像识别、视频处理、计算机视觉、语音识别、自然语言处理乃至脑电波分析等众多领域取得了重大突破。CNN 的设计灵感源于生物的视觉系统，能够自动并有效地从原始数据中学习到的复杂的特征表示，使其在处理高维度数据时表现出了卓越的性能。

CNN 可以根据处理数据的维度不同分为三种主要类型：

一维卷积神经网络（1D-CNN）：主要应用于序列数据的处理，如时间序列分析、自然语言处理等领域，通过分析文本序列中的局部模式来提取关键信息。

二维卷积神经网络 (2D-CNN)：广泛应用于图像处理和计算机视觉任务，通过学习图像中的空间层次结构来实现对象识别和分类。

三维卷积神经网络 (3D-CNN)：用于视频内容分析和三维图像处理，能够捕捉时间维度上的动态信息，如人物行为识别等。

CNN 主要包括卷积层、池化层和全连接层，它们共同构成了 CNN 的基本结构。如图 2.5 是 CNN 的结构图。

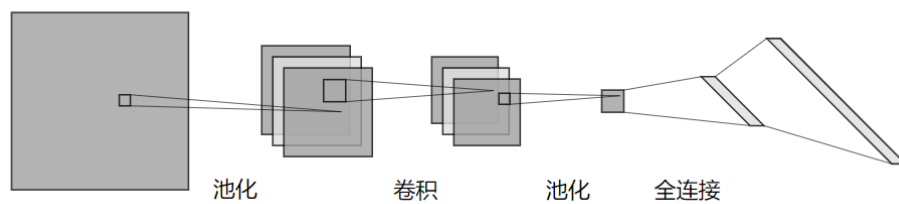


图 2.5 CNN 结构图

卷积层：利用卷积核提取输入数据的局部特征，其中卷积核的尺寸和数量是影响模型性能的关键参数。通过逐个元素的相乘后求和的方式，卷积操作提取了数据的局部空间特征，而非线性激活函数如 ReLU 则引入了非线性因素，增强了模型的表达能力。

池化层：在卷积层之后通常会跟一个池化层，其主要作用是进行下采样，降低特征维度和计算量，同时保持关键信息。最大池化和平均池化是两种常见的池化策略，它们分别通过取局部区域的最大值和平均值来实现降维。

全连接层：在 CNN 的最后几层通常包含全连接层，它将前面卷积层和池化层学到的特征进行综合和高度非线性变换，输出最终的分类或回归结果。在多分类任务中，softmax 函数常用于输出层，将多维输出映射成概率分布。

尽管 CNN 在多个领域展现了巨大的应用潜力和优异的性能，但其训练过程对计算资源的需求较高，特别是对于含有大量参数的深层网络。此外，为了防止模型过拟合，通常需要采取数据增强、正则化等措施。随着研究的深入和技术的发展，CNN 正不断被优化和改进，以适应更多复杂任务的需求，推动人工智能技术向前发展。

2.4 集成学习策略

集成学习是一种强大的机器学习策略，它通过构建并组合多个学习器（通常称为弱学习器）来解决同一问题，目标是通过集成的方式提高整体模型的预测性能。集成学习的基本思想是多个模型联合比单一模型单独工作时更为有效。集成学习主要分为三类：Bagging、Boosting 以及 Stacking。

Bagging^[43]，即 Bootstrap Aggregating 的简称，是一种通过组合多个模型减少方差，提高稳定性和准确性的集成方法。其核心思想是使用自助采样（Bootstrap sampling）从原始数据集中随机抽取多个子集，然后对每个子集独立地训练出一个模型，最终通过对所有模型的预测结果进行平均或多数投票的方式来决定最终的输出。典型的 Bagging 算法包括随机森林。

Boosting^[44] 方法关注于提高弱学习器的性能。它通过顺序地训练模型，每一步都会根据前一步的表现调整数据的权重，使得之前模型预测错误的数据在后续模型中获得更多的关注。这样，Boosting 方法能够不断减少模型的偏差。Boosting 的目标是将多个性能相对较弱的模型组合成一个强大的模型。典型的 Boosting 算法有 AdaBoost、Gradient Boosting Machine (GBM) 和 XGBoost 等。

Stacking（堆叠泛化）^{[45][46][47]} 是一种用于集成多种不同模型的方法。不同于 Bagging 和 Boosting 主要关注单一类型模型的集成，Stacking 通常涉及多种类型的模型。其基本思想是首先使用原始数据集训练多个不同的模型，然后再将这些模型的预测结果作为新的特征，用来训练一个新的模型（通常称为元模型或二级模型），以此来预测最终的目标。如图 2.6 是 Stacking 的架构图。

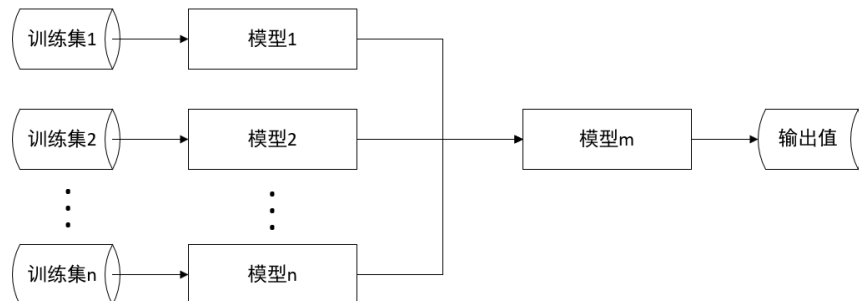


图 2.6 Stacking 架构图

集成学习的主要优势在于它能够显著提高预测性能，尤其是对于那些单一模型难以处理的复杂问题。通过组合多个模型，集成学习能够降低过拟合的风险，提高模型的鲁棒性和泛化能力。然而，集成学习也面临着一些挑战，包括计算成本的增加、模型训练和调参的复杂性，以及对于某些类型的数据集或问题，集成方法可能并不总是能够带来性能的提升。

2.5 本章小结

本章深入探讨了恶意代码检测的理论基础和实际应用方法。首先，明确了恶意代码的定义与分类，并概述了当前主流的恶意代码检测技术。其次，分析了恶意代码采用的动态执行和内存加密等免杀技术，这些技术旨在规避检测机制。此外，本章还简要介绍了在本文中应用的机器学习和深度学习模型，这些模型对于自动识别和分类恶意代码起到了重要的作用。最后，讨论了集成学习策略的应用，强调了通过结合多个模型来提高准确率的重要性。这些理论知识和技术方法为提出的恶意代码检测方案提供了理论支持。

第三章 基于行为特征的恶意代码检测算法

近年来，Windows 终端安全面临的挑战日益严峻，尤其是由于一系列行为检测绕过技术的出现。这些技术可以规避传统的安全监控机制，对终端安全构成了重大威胁。例如，利用直接系统调用技术，恶意代码可以绕过常规 API 调用，直接与操作系统内核交互，从而使基于 Inline Hook 的沙箱 API 收集机制失效。这种绕过机制进而影响了依赖 API 序列分析的机器学习检测模型，降低了其识别恶意代码的准确性。因此，本文针对 Windows 终端上采用新型免杀技术的恶意代码行为特征，融合多个检测角度，提出了一种有效的检测算法。下面将详细介绍检测算法的整体结构和算法设计。

3.1 检测算法的基本框架

本章提出的检测算法分为两个核心部分：一是针对动态加载阶段的行为特征，目的是在攻击成功前能够有效识别出潜在的隐蔽恶意行为；二是针对内存运行阶段的行为特征，旨在检测无文件攻击或已经驻留在内存中的恶意代码。如图 3.1 是检测算法的基本框架。

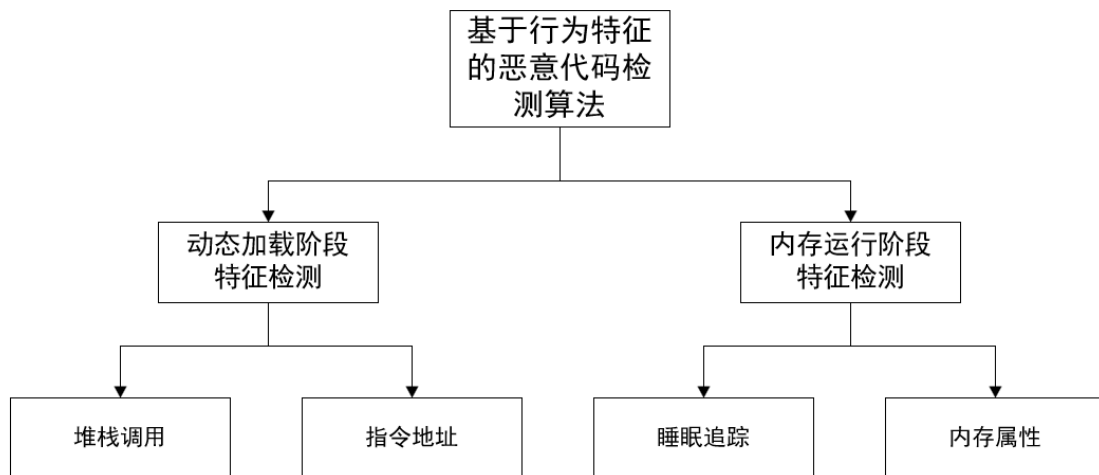


图 3.1 检测算法的基本框架

3.2 动态加载阶段的特征检测

3.2.1 动态加载阶段的行为特征分析

当前动态加载阶段主流的免杀技术是直接或间接系统调用(Direct/Indirect Syscall),也称为用户态挂钩绕过。这种技术通过构造汇编代码直接调用内核态函数,绕过了 ntdll.dll 的常规路径。然而,此方法会留下一些明显的行为特征。在标准程序执行过程中,系统调用流程通常是:主程序模块->kernel32.dll->ntdll.dll->syscall。这意味着,当内核态执行完毕并返回用户态时,返回地址应位于 ntdll.dll 的地址范围内。相反,如果是恶意程序直接执行系统调用,用户态的返回指令指针(RIP)将位于主程序模块内,而不是 ntdll.dll 地址范围。这个差异成为了检测用户态挂钩绕过技术的一个准确标志。通过识别这种异常的返回地址模式,可以有效地检测并防御此类免杀技术。如图 3.2 是正常程序系统回调的示意图。

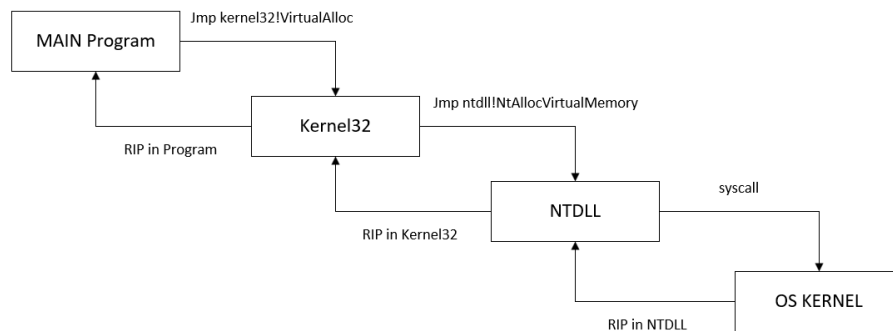


图 3.2 正常程序系统回调示意图

3.2.2 动态加载阶段的特征检测算法设计

在动态加载阶段,特征检测算法主要从系统调用的堆栈调用和指令地址两个角度进行。首先,通过获取从内核态到用户态的回调并分析调用堆栈的合法性来判断是否存在恶意行为。然后,利用 ETW 对系统调用的指令地址进行监控,通过与已知的合法地址进行比对来识别潜在的异常或恶意模式。将这两者结合应用,可以更全面地检测动态加载阶段的恶意代码。

1. 从堆栈调用角度进行检测

该步骤主要利用 Nirvana^[48] 动态回调框架和 StackWalk64 函数,其中 Nirvana

框架主要用于在用户态下监视和控制正在运行的进程。利用 Nirvana 框架，可以识别出未通过已知有效路径从内核态转换回用户态的系统调用，从而判断程序是否采用了直接系统调用技术。此外，结合 StackWalk64 函数，可以在 Nirvana 框架检测到异常系统调用时，遍历堆栈调用上下文，通过验证堆栈的合法性来识别间接系统调用技术。

在 Windows 操作系统中，应用程序通常依赖 Win32 API（例如，由 kernel32.dll 提供的函数）来执行各种系统级任务，如文件操作、内存管理和进程控制。然而，kernel32.dll 及其他 Win32 API 实际并不直接执行这些底层操作。相反，它们充当封装层，通过调用 ntdll.dll 中的 NT 函数（也称为 Native API）与 Windows 内核交互，从而实现系统调用的功能。理想的防御机制应当能够允许所有来自合法源的系统调用，同时阻止那些来自恶意源的调用。虽然直接或间接的系统调用在功能上可能与合法调用相似，但它们通常出现在不预期的执行点。通过监控从内核态回到用户态的转换点，可以有效地检测并防止这种潜在的恶意活动。当内核完成一个请求的操作并将结果返回给 ntdll.dll，最终传递回原始 Win32 API 调用者的过程中，操作系统需确保能够正确地恢复和访问上下文信息。如图 3.3 是 API 合法调用的流程图。

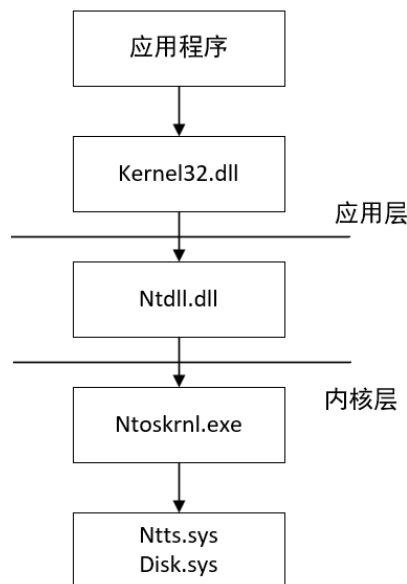


图 3.3 API 合法调用流程图

因此可以通过 Nirvana 框架来监测和检测从内核到用户态的转换过程中的异常行为，特别是那些可能由恶意代码触发的非正常系统调用行为。在系统调用发出后，R10 寄存器通常会包含用户态要执行的第一条指令的地址，这几乎总是一条返回指令。通过验证这个地址的完整性，Nirvana 能够检测到直接或间接系统调用的存在，进而判断系统调用是否在已知的、安全的位置返回到用户态。这个位置通常是 ntdll.dll 或 win32u.dll 中的导出函数，而不是 .text 部分中某未知模块的内存页。通过设置一个预定义的“允许列表”检查发起系统调用的模块，以此来识别和防范未授权或恶意的系统级操作。当应用程序通过 ntdll.dll 发起系统调用时，当前线程的上下文（包括寄存器状态等）会被保存，通常是通过 RtlCaptureContext 或类似机制自动完成的，或者在进行系统调用转换时显式地保存相关信息。系统调用完成后，执行流程返回到用户态，在 ntdll.dll 中的一个特定点恢复执行，通常是系统调用指令后的下一条指令。此时，内核通过之前保存的上下文信息来恢复调用线程的状态，包括寄存器状态和栈指针，确保线程可以正确地继续执行。

通过上述方法可以来有效监控和防御直接系统调用，进而防止恶意代码的动态加载。而间接系统调用，不同于直接通过系统调用号执行的系统调用，通常是通过 ntdll.dll 中的跳板或存根函数来间接执行的。这种调用方式不仅在合法软件中常见，也经常被恶意代码利用来避免直接检测。如图 3.4 是间接系统调用的示意图。

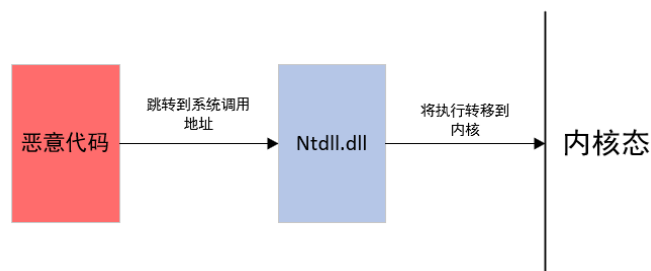


图 3.4 间接系统调用示意图

为了检测间接系统调用，本算法增添 StackWalk64 函数则可以用于追踪间接系统调用的行为。通过 StackWalk64 函数在被监视的进程中追踪调用堆栈，遍历各个函数调用链。这一过程中，会收集函数的返回地址和调用者的帧指针信息。通过分

析堆栈帧的程序计数器（即 `stack.AddrPC.Offset` 指向的地址），识别相关模块名称，可以判断系统调用流程是否遵循从主程序模块经过 `kernel32.dll` 到 `ntdll.dll`，最终执行 `syscall` 的标准路径。这样，从堆栈调用的角度，不仅可以追踪系统调用的来源和流程，还能帮助识别和防范通过非常规路径执行的潜在恶意调用。

堆栈调用角度检测的核心代码如下：

Algorithm 1 堆栈调用角度检测恶意代码

Input: *context, process, thread*

Output: 直接或间接系统调用的检测输出

```

1: Define RIP_CHECK(Rip, BaseAddress, Size) as (Rip < (BaseAddress +
   Size)) and (Rip > BaseAddress)
2: if RIP_CHECK(context->Rip, NtdllAddress, NtdllSize) and not NtdllAddress
   then
3:   printf("[%] Direct system call found.\n");
4: elseif RIP_CHECK(context->Rip, W32Address, W32Size) and not W32Address
   then
5:   printf("[%] Direct system call found.\n");
6: end if
7: while StackWalk64(process, thread, &stack, &context) do
8:   moduleName ← GetModuleNameFromAddress(stack.AddrPC.Offset)
9:   moduleNames.push(moduleName)
10: end while
11: secondLastModuleName ← moduleNames[-2]
12: if secondLastModuleName ≠ "kernel32" then
13:   printf("[%] Indirect system call found.\n");
14: end if

```

2. 从指令地址角度进行检测

随着技术的发展，新型免杀技术通过使用硬件断点来伪造合法的堆栈调用，使得攻击者可以在用户态下规避传统的监测手段。该方法涉及一系列的初始化步骤，包括获取当前线程的句柄以设置断点、获取 `ntdll` 模块的句柄、在 `kernel32.dll` 中寻找用于返回主程序的 `Gadget`，并注册异常处理程序以便在触发硬件断点时控制执行流。通过设置硬件断点并构造一个新的堆栈，该技术将找到的 `Gadget` 地址作为

返回地址，同时指定 RIP 为系统调用指令的地址，从而实现通过伪造的堆栈进行间接系统调用，绕过检测。这一流程模拟了从主程序模块到 kernel32.dll、ntdll.dll，最终到系统调用的执行路径。如图 3.5 是硬件断点的间接系统调用示意图。

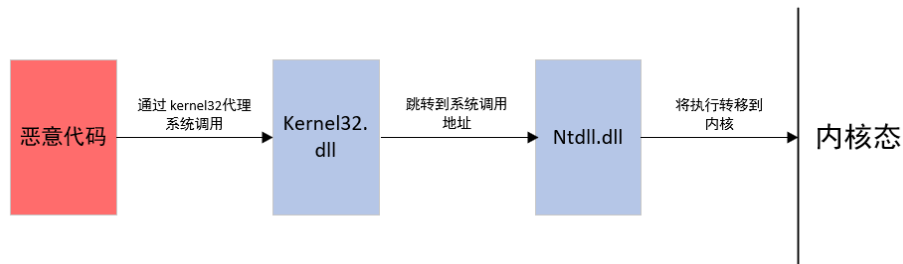


图 3.5 硬件断点的间接系统调用示意图

因此，攻击者可以通过硬件断点伪造合法的调用堆栈来使得堆栈调用角度的方法失效。面对这种情况，本算法另一个角度从指令地址出发，使用 ETW 获取内核中的提供程序捕获直接与间接的系统调用事件，并分析这些事件中包含的用户态地址调用跟踪信息，明确识别出触发系统调用的确切位置。这包括通过 ntdll.dll 执行的 syscall 指令地址，特别地，监控系统还能辨识出间接系统调用是否源自 ntdll.dll 内部的合法函数地址。此检测机制的核心在于利用 NT 函数的 syscall 汇编指令地址作为关键的检测标准。将这一指令 HOOK 之后，攻击者则无法正常访问这一地址，进而无法通过原始的 syscall 汇编指令来执行系统调用。通过对调用地址与 ntdll.dll 内特定函数地址的比较，来检测间接系统调用是否确实来源于 ntdll.dll 中的预定函数。这一过程是区分正常的系统调用和恶意行为的重要特征。

本算法通过 ETW 监控常被恶意代码利用 NT 函数。在启用 Microsoft-Windows-Kernel-Audit-API-Calls 提供者并设置 EVENT_ENABLE_PROPERTY_STACK_TRACE 标志后，ETW 追踪并记录特定 API 调用事件的堆栈信息，重点关注内存分配与线程创建等事件。配置的回调函数使得对捕获的事件进行返回 callStack 中，尤其在直接与间接系统调用的行为识别上。接着，算法通过对内存分配与线程创建等 NT 函数的 syscall 指令设置 HOOK，来防止攻击者利用已知模式搜索这些 syscall 指令的具体位置，并跳转至这些地址执行系统调用。

检测算法涵盖两个核心函数：DetectDirectSyscall 和 DetectIndirectSyscall。DetectDirectSyscall 函数旨在识别恶意程序直接发起的系统调用，这类调用绕过了操作系统的标准调用机制。通过匹配系统调用发起的模块名称与预定义允许模块列表，此方法能有效标记未授权的直接系统调用，揭示潜在恶意行为。另一方面，DetectIndirectSyscall 函数用于间接系统调用的检测，这类调用可能由恶意代码隐藏其真实来源和行为。该函数通过分析系统调用的来源模块及与 ntdll.dll 中预期地址的偏移量来判断系统调用的合法性。如果系统调用的实际地址与预期地址间的偏移量超出合理范围，则将该间接系统调用标记为非法，暴露恶意代码的存在。DetectDirectSyscall 和 DetectIndirectSyscall 的结合不仅能检测直接或间接系统调用，还能识别通过硬件断点伪造堆栈的系统调用。

指令地址角度检测的核心代码如下：

Algorithm 2 指令地址角度检测恶意代码

Input: *processHandle*, *callStack*, *detectedSyscall*

Output: 直接和间接系统调用的检测输出

```

1: Initialize: AllowedModules = {"ntdll.dll", "win32u.dll"}
2: function DetectDirectSyscall(processHandle, callStack)
3:   module  $\leftarrow$  GetModuleName(processHandle, head(callStack))
4:   if module NOT in AllowedModules then
5:     printf("[%] Direct system call found.\n");
6:   end if
7: end function
8: function DetectIndirectSyscall(processHandle, callStack, detectedSyscall)
9:   module  $\leftarrow$  GetModuleName(processHandle, head(callStack))
10:  if module = "ntdll.dll" then
11:    expectedAddress  $\leftarrow$  GetProcAddress("ntdll.dll", detectedSyscall)
12:    actualOffset  $\leftarrow$  Offset of head(callStack) from module base
13:    expectedOffset  $\leftarrow$  Offset of expectedAddress from module base
14:    if Not in expected range(actualOffset, expectedOffset) then
15:      printf("[%] Indirect system call found.\n");
16:    end if
17:  end if
18: end function

```

3.3 内存运行阶段的特征检测

3.3.1 内存运行阶段的行为特征分析

在内存执行阶段，主要检测的是内存中的恶意代码。当前主流的 C2 远程工具通常在不执行命令时进入睡眠状态，并在执行命令时要求内存具备可执行属性。远控工具默认会设置等待时间，并允许通过 Sleep 命令调整该时间。这一策略有效控制了恶意代码的通信间隔。在恶意代码的实现中，系统的 Sleep 函数用于触发睡眠状态，而服务端采用消息队列机制来存储待执行的命令。当恶意代码与服务端建立连接时，它会从队列中读取并执行命令。在这个过程中，当线程处于 Sleep 睡眠状态时，其返回地址通常会指向内存中的恶意代码。因此，通过检查可疑进程中线程的返回地址，就可以轻易地发现植入的恶意代码。

并且内存属性的变化也是识别内存恶意代码的关键行为特征。内存可以设置为可读、可写或可执行，并且被分类为映像提交或私有提交。例如，从磁盘加载的文件（如 EXE 或 DLL）创建的内存被视作“映像提交”，而通过 API 如 VirtualAlloc 动态分配的内存则归为“私有提交”。此类技术的核心在于扫描内存页，以识别可执行内存和私有提交内存。

3.3.2 内存运行阶段的特征检测算法设计

对于运行内存中的恶意代码，本算法通过睡眠追踪和内存属性两个角度进行检测：

1. 从睡眠追踪角度进行检测

该步骤主要在受检查线程的调用堆栈中查找 Shellcode 引用。典型的可疑线程指标包括延迟执行状态，其中恶意代码大部分时间处于睡眠状态，周期性唤醒以执行其任务。实现这种行为的常见方法是通过调用如 Sleep 或 WaitForSingleObject 这样的 Windows API，导致线程进入等待状态。这样的线程调用堆栈将包含对 KernelBase.dll!WaitForSingleObjectEx 和 ntdll.dll!ZwWaitForSingleObject 的调用。当这种行为与其他指标结合，如调用堆栈中的虚拟内存调用时，可以揭示恶意代码的行为模式。如图 3.6 是指向恶意代码的返回地址示意图。

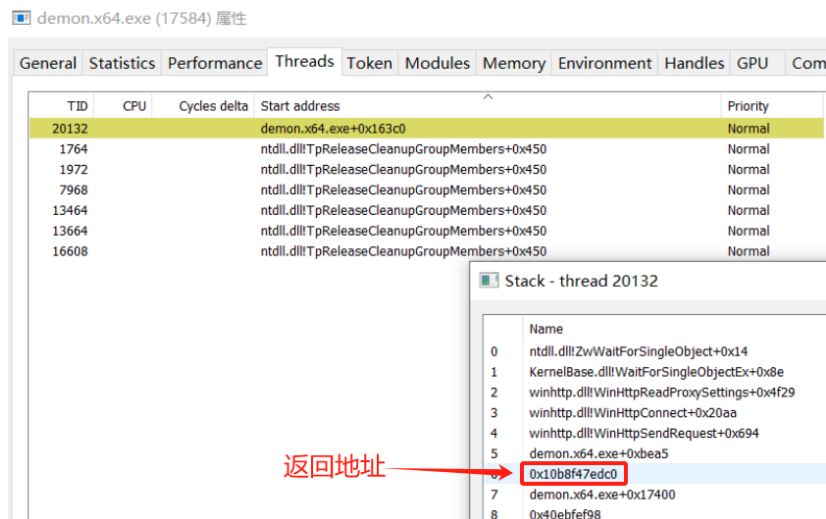


图 3.6 指向恶意代码的返回地址示意图

针对这些特征，本算法引入了一种用于检查特定进程中线程的调用堆栈的方法，旨在识别异常情况或模块篡改行为。该函数首先获得目标进程和线程的句柄，接着获取线程的上下文信息。随后，初始化符号引擎以解析调用堆栈中的符号信息。利用 `StackWalk64` 函数遍历线程的调用堆栈，对每个堆栈帧进行检查。尝试通过 `SymGetModuleInfo64` 获取模块信息，判断指令指针是否对应磁盘上的文件。如果无法对应，则将其标记为未知模块并记录相关信息，表示存在异常调用堆栈。若能获取到模块信息，则记录模块的符号名称和文件名，进而检查模块是否遭到篡改。此外，本算法还引入了对 `WaitForSingleObject` 调用的检测，用于识别由 APC（异步过程调用）发起的 `WaitForSingleObject` 调用，从而进一步增强对延时执行线程的监测能力。这是因为等待 `WaitForSingleObject` 函数返回的线程通常处于 `UserRequest` 状态，而非 `Sleep` 的 `DelayExecution` 状态。

为减少对 `UserRequest` 状态的误报，打开指定的线程，获取其上下文信息，尤其是栈指针（RSP）的状态。通过线性探测栈空间，检查每个栈地址是否指向关键的系统函数入口点。如果栈中的地址与 `KiUserApcDispatcher` 函数的有效地址范围匹配，说明可能存在 `Foliage` 行为；如果栈地址直接指向 `CreateTimerQueue` 函数的地址，表明可能存在 `Ekko` 行为。

从睡眠追踪角度检测恶意代码的核心代码如下：

Algorithm 3 睡眠追踪角度检测恶意代码**Input:** *hProcess, hThread***Output:** 异常内存区域的检测输出

```

1: if wait: DelayExecution then
2:   GetThreadContext(hThread, &context)
3:   stackframe.AddrPC.Offset  $\leftarrow$  context.Rip
4:   StackWalk64(hProcess, hThread, &stackframe, &context,
     SymFunctionTableAccess64, SymGetModuleBase64
5:   result  $\leftarrow$  SymGetModuleInfo64(hProcess, stackframe.AddrPC.Offset)
6:   if not result then
7:     print("DelayExecution state identified anomalous memory region.")
8:   end if
9: else if wait: UserRequest then
10:  ReadProcessMemory(hProcess, &stackAddr)
11:  if stackAddr = pKiUserApcDispatcher then
12:    print("UserRequest state identifies anomalous memory regions.")
13:  end if
14:  if stackAddr = pCreateTimerQueue then
15:    print("UserRequest state identifies anomalous memory regions.")
16:  end if
17: end if

```

2. 从内存属性角度进行检测

为了有效识别和预防内存中的恶意代码，分析操作系统所分配给进程的内存页权限显得尤为重要，尤其是在通过内存属性检测 Shellcode 方面。常见的恶意代码，如使用 Shellcode 的攻击，通常会尝试在拥有执行权限的内存区域注入其代码。鉴于大部分应用程序的内存页无需同时拥有写和执行权限，因此，检测具备 PAGE_EXECUTE_READWRITE（可读、可写、可执行）权限的内存页成为了一种既关键又准确的方式，用于识别潜在的恶意活动。

本算法实现了检查线程地址是否具有可执行属性的函数，通过检测指定地址处线程的内存是否具有执行代码的能力，来找寻潜藏在内存中的恶意代码。使用 ZwQueryVirtualMemory 函数查询指定地址的内存基本信息，检查内存区域的类型

和保护属性。如果内存区域不属于 MEM_IMAGE 类型（即不是由加载的模块分配的内存），并且具有执行权限（如包含 PAGE_EXECUTE、PAGE_EXECUTE_READ、PAGE_EXECUTE_READWRITE 或 PAGE_EXECUTE_WRITECOPY 等），则可能指示存在未知或可疑的内存区域，这可能是通过 VirtualAlloc 分配以规避检测。

另一方面，本算法还通过利用 ETW 事件来识别因内存保护属性波动而隐藏恶意代码的不可执行区域。使用 KrabsETW 库建立用户态的 ETW 跟踪会话，并配置了 Microsoft-Windows-Threat-Intelligence 作为事件提供者，用于监控具有 ProtectVM 关键字的内核事件，这些事件标记了内存保护级别的关键变化。通过设定事件过滤器，仅捕获与内存保护状态变化相关的事件。定义的回调函数在事件触发时分析事件记录，提取进程标识符、内存基址、当前和之前的内存属性。比较这些内存属性可检测到从不可执行到可执行或从可写到不可写的关键状态变化。系统在运行时成功监控指定的内存保护事件，并能对关键保护状态变化进行有效记录和响应。当内存页多次规律性地从可写变为不可写或从不可执行变为可执行时，表明该内存区域存在异常行为。如图 3.7 是内存可执行属性更改的示意图。

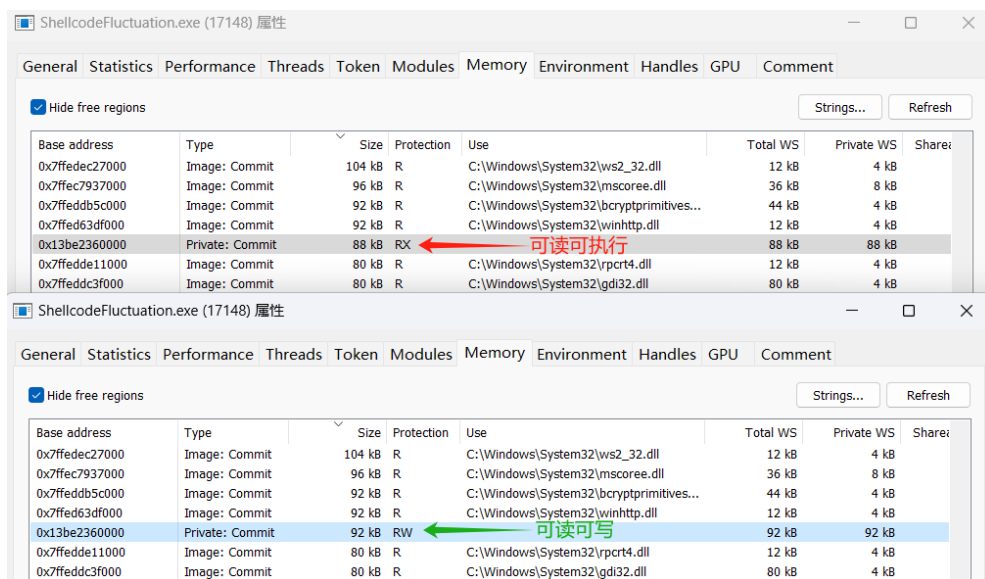


图 3.7 内存可执行属性更改示意图

通过结合 YARA 扫描技术，本算法针对处于可执行状态的可疑内存区域进行内存转储，从而获取了转储文件。利用 YARA 对这些转储文件进行扫描，通过其精确的模式匹配和规则识别，有效提高了对已知威胁的检测效果。这样方法不仅

拓宽了对恶意代码检测的维度，尤其针对那些利用内存保护属性变化来隐藏自身的恶意代码，从而增强了对潜在恶意行为的防御能力。

从内存属性角度检测恶意代码的核心代码如下：

Algorithm 4 内存属性角度检测恶意代码

Input: 进程标识符和相应的内存区域信息

Output: 异常内存区域的检测输出

```

1: for each memory region mbi in process identified by ProcessID do
2:   if mbi.Type  $\neq$  MemImage then
3:     hasExecuteFlag  $\leftarrow$  mbi.AllocationProtect has any of
4:       PageExecute, PageExecuteReadWrite, PageExecuteRead
5:     if hasExecuteFlag then
6:       dumpFile  $\leftarrow$  MiniDump(ProcessID)
7:     end if
8:   end if
9: end for
10: Create ETW provider tiProvider with Microsoft-Windows-Threat-Intelligence
11: Set tiProvider to monitor events with ProtectVM keyword
12: if !IsExecutable(oldProtection) and IsExecutable(newProtection) then
13:   protectionChangeCount[ProcessID]  $+=$  1
14:   if protectionChangeCount[ProcessID]  $\geq$  3 then
15:     dumpFile  $\leftarrow$  MiniDump(ProcessID)
16:   end if
17: end if
18: yaraResult  $\leftarrow$  YARA Scan(dumpFile)
19: if yaraResult = Malicious then
20:   Print detection message with all addresses and process details
21: end if

```

3.4 实验与结果分析

3.4.1 实验环境

本文的实验环境配置具体如表 3.1 所示。

表 3.1 实验环境配置

项目	参数
操作系统	Windows 10 64 位
编程语言	C++
开发软件	Visual Studio 2022
虚拟机	VMware® Workstation 17 Pro 17.0.0 build-20800274
内存	16GB
硬盘	256GB SSD + 1TB HDD

本章设计的检测算法使用 C++ 语言实现，并对动态加载与内存执行两个阶段进行了实验验证。实验样本主要来源于 GitHub 上的开源项目，这些项目代码用于生成实验的样本。如表 3.2 展示了相关直接或间接系统调用的开源项目概述。

表 3.2 直接或间接系统调用开源项目介绍

项目名称	项目介绍
HellsGate	HellsGate 通过 PEB 定位 ntdll.dll，搜索导出表以匹配 syscall stub 字节码，获取系统调用号并直接构造并执行 syscall 指令。
FreshyCalls	FreshyCalls 采用间接系统调用的方式，通过将执行流跳转 (jmp) 至 syscall 指令的地址，实现在 ntdll 中执行系统调用。
SysWhispers3	SysWhispers3 采用动态搜索方法在 ntdll.dll 中定位系统调用指令，避免了可能被 dll 中的内联钩子破坏的静态偏移量问题。
HWSyscalls	HWSyscalls 通过设置硬件断点，在 kernel32 上创建合成跳板，以此来实现间接系统调用并伪造正常的执行堆栈。

通过采用内存规避技术，生成用于内存运行阶段实验的免杀样本。如表 3.3 展示了相关内存规避开源项目的简介。

表 3.3 内存规避开源项目介绍

项目名称	项目介绍
ShellcodeFluctuation	ShellcodeFluctuation 通过循环加密与解密 Shellcode 内容，并在可读写与可执行内存保护状态之间切换。
ThreadStackSpoofers	ThreadStackSpoofers 通过修改线程返回地址，隐藏调用堆栈，以防止检测到执行恶意代码的线程。
Foliage	Foliage 通过利用 NtContinue API 调用控制线程在睡眠期间的执行流程，以恢复线程的执行上下文。
Ekko	Ekko 通过 CreateTimerQueue 创建计时器队列，在回调中调用内存权限修改、内存加解密与延迟执行的功能。

3.4.2 结果分析

本文主要进行了两项实验。动态加载阶段的实验旨在攻击成功前阶段识别并阻止恶意代码，提供前期预警。内存运行阶段的实验则针对已绕过初始安全防御、可能在内存执行恶意活动的代码，通过识别和清除这些威胁来保护终端安全。

(1) 第一项实验的评估指标为直接或间接系统调用的成功与否。本实验分别使用 HellsGate、SysWhispers3、FreshyCalls 和 HWSyscalls 技术对本章两个检测角度进行实验。实验结果如表 3.4 所示。

表 3.4 动态加载阶段检测算法测试结果

	堆栈调用	指令地址	总体
HellsGate	检出	检出	检出
SysWhispers3	检出	检出	检出
FreshyCalls	检出	检出	检出
HWSyscalls	未检出	检出	检出

实验结果表明，第一种方法能够有效检测直接和间接的系统调用，但它存在一

定的局限性，尤其是在面对经过堆栈伪造的硬件断点间接系统调用（HWSyscalls）时，此方法未能成功识别这类高级技术。相比之下，第二种方法采取了更为深入的检测机制，通过监控 syscall 指令的存根位置，不仅可以识别常规的直接和间接系统调用，还能有效检测那些经过堆栈伪造进行的间接系统调用。这种方法的应用扩展了系统调用检测的范围，提高了对复杂攻击手段的识别能力。

（2）第二项实验针对隐藏于内存中的恶意代码。实验将从内存属性和睡眠追踪两个角度对 ShellcodeFluctuation、ThreadStackSpoofers、Foliage 和 Ekko 技术进行检测。（实验利用 BYOVD 技术启用 PPL，模拟收集 Microsoft-Windows-Threat-Intelligence 事件）实验结果如表 3.5 所示。

表 3.5 内存运行阶段检测算法测试结果

	睡眠追踪	内存属性	总体
ShellcodeFluctuation	检出	检出	检出
ThreadStackSpoofers	检出	检出	检出
Foliage	检出	检出	检出
Ekko	检出	检出	检出

实验结果表明，所采用的方法能够有效地检测和区分不同类型的恶意代码行为。具体而言，ShellcodeFluctuation、ThreadStackSpoofers、Foliage 以及 Ekko 在内的所有测试恶意代码都在睡眠追踪和内存属性分析两个方面成功被检出。这些结果验证了该方法在处理不同恶意策略时的可靠性和有效性，每种恶意代码均在所有测试维度中被一致地识别，展示了其较好的适用性与识别能力。

3.5 本章小结

本章提出了基于行为特征的恶意代码检测算法。在动态加载阶段，本文从堆栈调用和指令地址两个角度出发，在进程或线程创建之前，有效识别出恶意模块。而在内存运行阶段，从睡眠追踪和内存属性两个角度出发，睡眠追踪则能够追溯

恶意代码的执行路径，包括通过复杂调用链尝试逃避检测的行为，而内存属性分析能够识别那些试图隐藏其行为的恶意代码。

通过实验验证，检测算法在动态加载和内存运行两个阶段都显示出较好的检测效果，能够有效地识别并阻断恶意代码的攻击行为。这两大类检测技术的综合应用，不仅能在早期阶段预防恶意代码的成功执行，还能在恶意代码绕过初级安全防御后，通过内存分析技术准确识别并清除内存中的恶意行为，为终端安全提供了双重保障。

第四章 基于 Stacking 集成学习的恶意代码检测模型

在终端安全领域，面对未知变体恶意代码对传统规则检测方法所构成的挑战，本文提出了一种融合 XGBoost 与改进 TextCNN 的恶意代码检测模型。将恶意代码中的 API 调用序列为文本序列，本文采纳了 Stacking 集成学习策略，利用 XGBoost 和改进 TextCNN 基模型进行第一层训练并得到元特征，进而使用 XGBoost 元模型进行第二层深入训练。通过融合各基模型的预测结果，提高了对恶意代码未知变体的检测能力，并因此增强了模型的泛化能力和准确性。以下将具体介绍这一检测模型的基本框架及其训练过程。

4.1 检测模型的基本框架

本章介绍基于 Stacking 集成学习的恶意代码检测模型，其中主要包括数据预处理阶段、XGBoost 与改进 TextCNN 的基模型训练、采用 Stacking 集成学习的元模型训练以及模型预测与评估阶段。如图 4.1 是检测模型的基本框架。

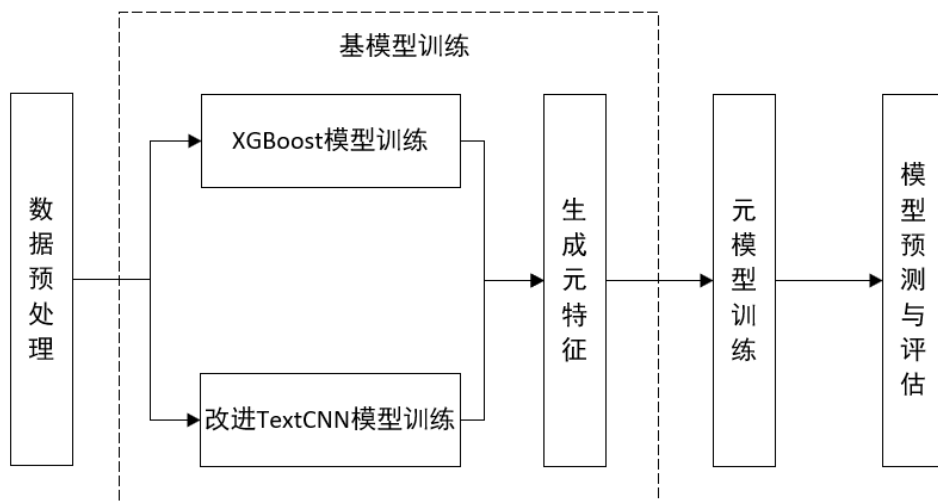


图 4.1 检测模型的基本框架

(1) 数据预处理阶段：首先通过沙箱对样本的 API 序列进行获取，随后对这些序列进行去重处理，确保所获得的 API 序列的有效性从而降低了重复或无效 API

调用带来的计算负担。

(2) XGBoost 与改进 TextCNN 的基模型训练: 1. XGBoost 模型训练。XGBoost 在各种短距离 n -gram 信息的非线性表示方面表现出色, 同时具备高效的训练速度和处理大规模数据的能力。2. 改进 TextCNN 模型训练。改进 TextCNN 模型用于处理文本特征, 一维卷积神经网络能够捕获长距离 n -gram 信息, 有效提取文本数据中的深层特征。

(3) 采用 Stacking 集成学习的元模型训练: 本文采用 XGBoost 和 TextCNN 作为第一层的基模型, 旨在处理数据和特征, 以捕捉恶意代码检测的关键信息。这些模型独立训练, 所生成的预测结果被作为元特征, 供 XGBoost 算法作为元模型的输入。通过 Stacking 集成学习策略, 本方法有效整合了多模型信息, 增强了模型对未知变体恶意代码的识别能力, 并提高了检测准确率和模型的泛化能力。

(4) 模型预测与评估: 完成模型训练后, 使用训练好的基模型对测试集进行特征提取, 生成元特征, 这些元特征随后被送入训练好的元模型中进行最终预测。评估模型性能时, 采用多项评价指标 (如准确率、精确度、召回率、F1 分数等), 以全面评估模型的预测能力。

4.2 XGBoost 与改进 TextCNN 的基模型训练

4.2.1 XGBoost 模型训练

XGBoost 是基于梯度提升决策树的算法, 对原有的 GBDT 算法进行了优化和改进。与传统的 GBDT 相比, XGBoost 引入了二阶导数的考量, 不仅考虑了一阶导数 (即梯度) 而且还考虑了二阶导数 (即曲率)。这一改进有助于更准确地估计模型的损失减少量, 从而在训练过程中做出更合理的树结构调整。

XGBoost 算法通过增加决策树来最小化一个包含损失函数和正则项的目标函数。每一轮迭代中, 算法都会基于当前模型预测结果和实际标签之间的差异来计算梯度和二阶导数。这些信息随后被用来训练一个新的决策树, 以减少总体的预测误差。随着每棵新树的加入, 整个模型的预测性能逐渐提升, 直至迭代次数达到预定值或模型性能不再有显著改善。目标函数定义为:

$$L^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (\text{式 4-1})$$

其中, $L^{(t)}$ 代表在第 t 轮迭代后的总损失, l 是损失函数, 衡量模型预测 $\hat{y}_i^{(t-1)} + f_t(x_i)$ 与真实标签 y_i 之间的差异。 $f_t(x_i)$ 表示第 t 棵树对样本 x_i 的预测值, $\Omega(f_t)$ 为正则项, 控制模型复杂度, 以防过拟合。

XGBoost 的目标函数设计在拟合训练数据和控制模型复杂度之间取得平衡, 使其在多种机器学习任务上表现出色。目标函数进一步表示为:

$$\text{Obj} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (\text{式 4-2})$$

这里, Obj 是目标函数值, G_j 和 H_j 分别是第 j 个叶节点上所有样本梯度和二阶导数的总和, λ 是正则化参数, 控制模型复杂度, γ 是控制树复杂度的参数, T 是树的叶节点数。

在本节中, XGBoost 模型的训练流程包含以下步骤: 数据预处理、特征工程、模型训练、参数优化以及生成元特征。如图 4.2 是 XGBoost 模型训练的流程。

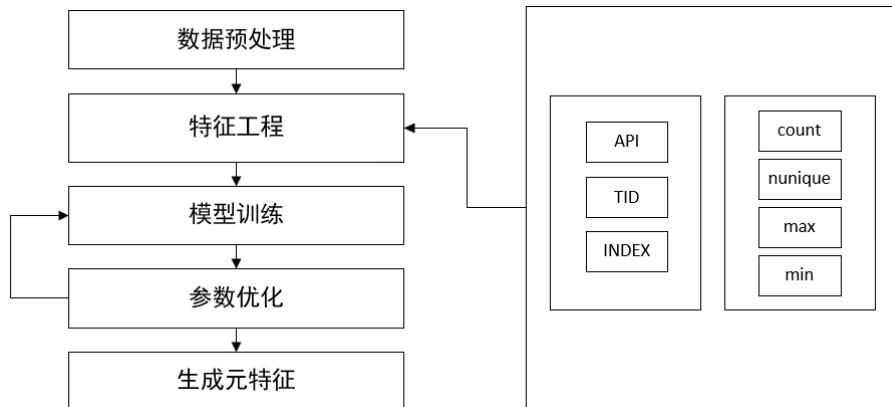


图 4.2 XGBoost 模型训练流程图

在数据预处理阶段, 主要通过沙箱技术收集恶意代码样本运行时数据, 这些数据包括有 API 调用序列 (API)、线程 ID (TID) 和调用索引 (INDEX)。在数据预处理阶段, 使用了数据清洗技术来处理缺失值和异常值, 去除数据中的噪声, 确保数据集的高质量和一致性, 为后续的特征工程和模型训练打下基础。

在特征工程阶段，本文采取了全局与局部两个层面的特征提取策略，并通过 2-Gram 算法从恶意代码 API 调用序列中提取特征。2-Gram 算法在实际应用中广泛使用，原因在于其计算量适中且能够在一定程度上体现词汇的共现度。2-Gram 算法特别关注 API 调用间的序列关系，通过分析相邻 API 调用的组合来捕捉 API 序列的局部模式和上下文信息。该算法不仅聚焦于单个 API 调用的特性，还探索了 API 调用之间的相互作用和顺序关系。例如，恶意代码可能以特定顺序执行 API 调用以达成恶意目的，2-Gram 算法能够通过识别这些调用序列模式来揭示潜在的恶意行为。如图 4.3 是生成 2-Gram API 序列的示意图。

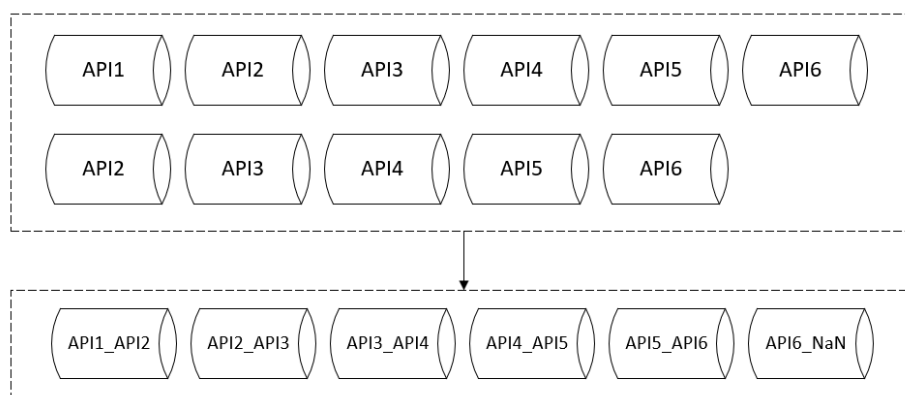


图 4.3 生成 2-Gram API 序列示意图

在全球层面，通过统计 API 调用次数（count）和 API 调用类别数（nunique），本文获得了每个样本整体行为的概览。这两个特征在区分恶意代码方面起到了重要作用，因为不同类型的恶意代码倾向于调用不同的 API 集合，而某些恶意代码类型，例如自我复制病毒，会频繁调用特定 API。因此，通过全局上的 count 和 nunique 特征可以推断出恶意代码的存在。这些全局特征反映了整个数据集中每个 file_id 对应的 API 调用模式的总体统计信息，其中 count 特征涉及计算每个 file_id 的 2-Gram API 序列出现的总次数，而 nunique 特征涉及计算每个 file_id 的 2-Gram API 序列中唯一元素的数量。

在局部层面，本文通过分析基于线程 ID 和调用索引的统计数据，如次数、唯一类别数、最大次数（max）和最小次数（min），超越了单一全局特征的限制，挖掘出了局部特征。例如，通过统计某线程内 API 调用的次数，可以揭示该线程程序

执行过程中的活跃程度；分析不同 API 调用的种类数可以了解线程执行过程中行为的多样性；最大次数和最小次数的统计有助于捕捉异常行为或特定的执行模式，如某些恶意代码可能会在短时间内重复大量特定 API 调用以执行恶意活动。这些局部特征针对每个文件的特定部分，提供了关于 API 调用序列的更精细的信息，包括基于 file_id + tid 的 count 特征和基于 file_id + index 的 max 和 min 特征。最后，通过采用 groupby 和 agg 等 Pandas 方法分别对全局特征和局部特征进行生成，所有生成的特征被合并到特征数据集中，为模型提供了信息以识别恶意代码。

在模型训练与参数优化阶段，主要对 XGBoost 模型的关键参数进行了调整和优化。这包括树的最大深度（max_depth），该参数控制模型复杂度，影响模型的学习能力和过拟合风险；学习率（learning_rate），决定每次模型更新的步长，对模型收敛速度和稳定性具有影响；数据集的采样率（subsample）和特征的采样率（colsample_bytree），这两个参数通过随机选择样本和特征来提高模型的多样性，从而增强模型的鲁棒性。再通过网格搜索与贝叶斯优化技术，在预定义的参数空间内系统地探索了各种参数组合，以确定能够产生最佳模型性能的参数配置。这确保模型在多个维度上的最优平衡，能够有效识别恶意代码，同时避免过拟合，保持良好的泛化能力。

在生成元特征阶段，通过网格搜索与贝叶斯优化技术得到的最佳模型性能参数配置，分别产生两组初始特征。这些初始特征的生成目的在于，它们随后将用于元模型的训练，以此来提高最终模型的预测准确性。XGBoost 模型参数配置如表 4.1 所示。

表 4.1 XGBoost 模型参数及其值

参数	参数值（网格搜索）	参数值（贝叶斯优化）
max_depth	8	9
learning_rate	0.1	0.0994
subsample	0.8	0.8952
colsample_bytree	0.9	0.7578

4.2.2 改进 TextCNN 模型训练

TextCNN 模型^[49]是由 Yoon Kim 基于传统的 CNN 架构专门为文本分类任务进行优化的。该模型在结构上与用于图像处理的 CNN 类似，但其输入层经过了特别设计以适配文本数据的特性。TextCNN 的核心优势在于其简洁的网络结构、较少的参数数量、低计算复杂度以及快速的训练能力，这些特性使其成为文本分类问题中的常用选择。

本文中的改进 TextCNN 模型基于传统 TextCNN 结构进行改进，在模型中加入自注意力层增强了模型对文本关键信息的识别和处理能力，并且采用标签平滑技术以减少模型对硬标签的依赖，进而增强了模型的泛化能力。该模型结构主要包括五个部分：嵌入层、卷积层、池化层、自注意力层以及全连接层。如图 4.4 是改进 TextCNN 模型的结构图。

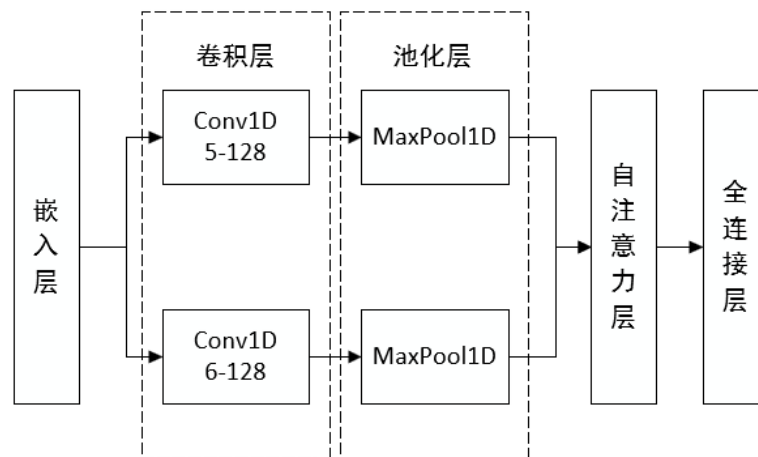


图 4.4 改进 TextCNN 模型结构图

(1) 嵌入层。该层在 TextCNN 模型中将文本转换为向量形式，关键在于深度学习处理。例如，API 调用序列被转换为向量矩阵，每行代表一个 API 调用。向量可以随机初始化或使用预训练的词向量，后者借助外部语料库加速学习并提高性能。

(2) 卷积层。该层通过不同大小的卷积核对嵌入层输出的向量进行卷积处理，旨在捕捉 API 调用序列中的局部特征。每种大小的卷积核都有两个实例，滑动窗口的大小决定了一次卷积覆盖的 API 数量，而卷积核的宽度与向量的维度相同。此

过程可理解为通过卷积操作从文本中提取关键信息，每个卷积操作输出一个特征值。

(3) 池化层。该层采用 1-Max 池化操作，针对每个卷积核生成的特征图提取最大值。若有 n 种不同尺寸的卷积核，生成 $2n$ 个特征图，则每个特征图通过池化操作转换为单个特征值，形成一个 $2n$ 维的特征向量。

(4) 自注意力层。该层位于卷积和池化层之后，采用自注意力机制以增强模型对序列中关键信息的捕获能力。自注意力机制允许序列中的每个元素评估其对序列其他元素的重要性，这是通过将序列中的每个元素同时视为查询 (Query)、键 (Key) 和值 (Value) 来实现的。自注意力计算的具体步骤如下：

首先，进行相似度评估：对于序列中的每个元素，计算其作为查询与所有元素作为键的点积，以评估元素间的相似度。这可以通过下式表达：

$$\text{Sim}_{i,j} = q_i \cdot k_j^T \quad (\text{式 4-3})$$

其中， q_i 是序列中第 i 个元素的查询向量， k_j^T 是序列中第 j 个元素的键向量的转置。

其次，权重归一化：使用 softmax 函数对每个元素的相似度分数进行归一化，以计算出对于每个查询，序列中各元素的权重分布。归一化后的权重表示为：

$$a_{i,j} = \frac{e^{\text{Sim}_{i,j}}}{\sum_{j=1}^N e^{\text{Sim}_{i,j}}} \quad (\text{式 4-4})$$

这确保了每个元素对自身的重要性评估转化为一个概率分布，其中权重总和为 1。

最后，加权求和：使用归一化后的权重对值向量进行加权求和，以获得每个元素的最终自注意力表示：

$$C_i = \sum_{j=1}^N a_{i,j} \cdot v_j \quad (\text{式 4-5})$$

这一步骤允许模型根据序列内部的上下文动态调整其对每个元素的处理，从而突出与当前任务最相关的信息。

(5) 全连接层。该层接收池化层输出的合并特征向量，通过全连接网络进行进一步的信息整合。随后，利用 Softmax 激活函数计算各分类的预测概率并完成分类。

在本节中，使用改进 TextCNN 模型对恶意代码进行深度学习检测。训练流程包含以下步骤：数据预处理、模型构建、模型编译、模型训练以及生成元特征。图 4.5 是改进 TextCNN 模型训练的流程。

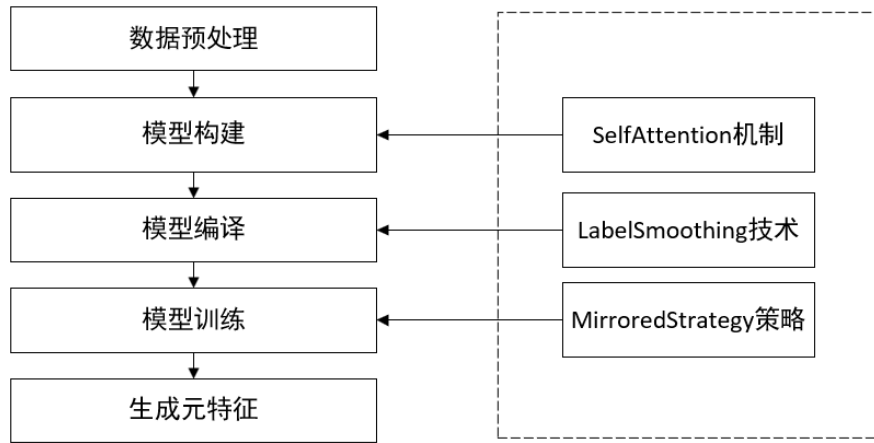


图 4.5 改进 TextCNN 模型训练流程图

在模型构建阶段，模型利用预处理后的 API 序列作为输入，其中输入层接收经预处理的文本序列，转换为嵌入向量形式，以 `max_len` 为序列固定长度，`embed_size` 为嵌入层维度，处理能力覆盖至 `max_cnt` 个 API。随后，通过一系列配置有不同核大小的一维卷积层，模型能够提取文本的局部特征。每个卷积层有 `num_filters` 个卷积核，采用了不同尺寸（5、6）的卷积核进行特征提取，并选用 ReLU 激活函数，以充分挖掘序列中的关键信息。此外，通过最大池化策略对特征进行降维，并通过 `concatenate` 操作整合了各卷积层的特征，以增强模型的表达能力。为提升模型对关键特征的识别能力，本文引入了自注意力机制，其目的是通过动态地调整权重，将模型的注意力集中在对分类决策重要的特征上。允许序列元素间基于相互关系动态调整表示权重。该机制通过 `MultiHeadAttention` 模块实现，单头维度由 `num_filters` 确定，其输出通过全局最大池化整合成固定长度向量。模型进一步将卷积与自注意力层输出合并，形成综合特征表示，随后经由全连接层加工。首个全连接层含 512 个神经元，采用 ReLU 激活函数，伴随 Dropout 层以强化模型对抗过拟合能力。最终，模型通过含 8 神经元的全连接层，应用 softmax 函数输出分类概率分布。

在模型编译阶段，为提高模型在不确定性数据面前的鲁棒性，本文引入标签平滑技术^[50]。通过自定义平滑分类交叉熵函数 `smooth_categorical_crossentropy`，结合标签平滑参数 `label_smoothing` 调整损失函数，模型训练过程中不仅关注正确类别预测的准确性，同时也优化对其他类别预测的处理。标签平滑技术的核心公式如下所示：

假设有一个标签平滑参数 ϵ ，对于一个具有 K 个类的分类任务，原始的硬标签 y 将被转换成一个新的标签 y' ：

$$y'_k = y_k \cdot (1 - \epsilon) + \frac{\epsilon}{K} \quad (\text{式 4-6})$$

其中， y_k 是原始标签中类别 k 的指标（0 或 1）， y'_k 是经过标签平滑处理后的新标签。

在这个公式中， $(1 - \epsilon)$ 是保留给正确类别的权重，而 $\frac{\epsilon}{K}$ 则均匀地分配给了所有类别，包括正确类别和错误类别。这样的处理方式使得模型对标签的预测不会过于自信，从而减轻了过拟合的风险，并提升了对不确定性数据的处理能力。

在自定义的平滑分类交叉熵函数中，损失计算公式调整为考虑了上述平滑后的标签 y' ，与模型预测的概率分布进行交叉熵计算：

$$L = - \sum_{k=1}^K y'_k \log(\hat{y}_k) \quad (\text{式 4-7})$$

其中， $\hat{y}_k$ 是模型对于类别 k 的预测概率。

通过这种方式，模型在训练时将更加注重学习数据的整体分布特征，而不是仅仅针对单个类别的预测，从而有效提升了泛化能力。

在模型训练阶段，使用 `MirroredStrategy` 策略，将模型构建与编译均在分布式训练策略 `strategy.scope()` 下进行，确保了多 GPU 环境下的数据并行性，加速训练过程。采用 Adam 优化器，以准确率评估性能。通过 TextCNN 与自注意力机制的结合、标签平滑正则化、及分布式训练策略，模型在处理大规模文本分类任务上展现较好的性能。最终，经过多次比较和验证，本文确定了模型的最佳参数配置，并证实了该模型在处理 API 序列的恶意代码检测任务中的有效性和准确度。改进 TextCNN 模型参数配置如表 4.2 所示。

表 4.2 改进 TextCNN 模型参数及其值

参数	参数值	参数	参数值
API 序列长度	10000	标签平滑参数	0.1
卷积核大小	5、6	自注意力层维度	128
卷积核数量	128	全连接层神经元数	512
卷积层激活函数	ReLU	优化函数	Adam
池化策略	最大池化	训练轮次	50

4.3 采用 Stacking 集成学习的元模型训练

在恶意代码检测上，XGBoost 模型虽然在处理结构化数据方面表现出色，但对特征工程高度依赖，对于动态行为特征的捕捉能力不足，且面临静态特征的局限性，这可能导致在处理复杂恶意代码特征时性能下降。同时，TextCNN 模型以其对文本序列的高效处理能力，在恶意代码的动态行为分析方面具有潜力，但其对大量训练数据的需求、处理非文本特征的挑战以及模型决策过程的低可解释性限制了其在实际应用中的广泛部署。因此，尽管这两种模型各有优势，但也存在着不可忽视的局限性，需要采用多模型融合或者特定策略来弥补各自模型的不足，从而达到更高的检测效率和准确率。

因此，本文采用了 Stacking 集成学习的方法来提高恶意代码检测的准确性和鲁棒性，选择 XGBoost 和改进 TextCNN 作为基础模型，同时利用 XGBoost 作为顶层的元模型进行最终预测，将 XGBoost 与 TextCNN 两种模型相结合。XGBoost 模型擅长各种短距离 n-gram 信息的非线性表征，具有高效的训练速率和处理大规模数据的能力。TextCNN 模型则在捕获长范围 n-gram 中的关键信息方面表现出独特的能力，有效提取文本数据中的深层次特征。通过综合这两种模型的优势，融合后的模型可在恶意代码检测的精度上取得提升，并在泛化性能上展现更优的表现。如图 4.6 展示了采用 Stacking 集成学习进行模型融合的流程。

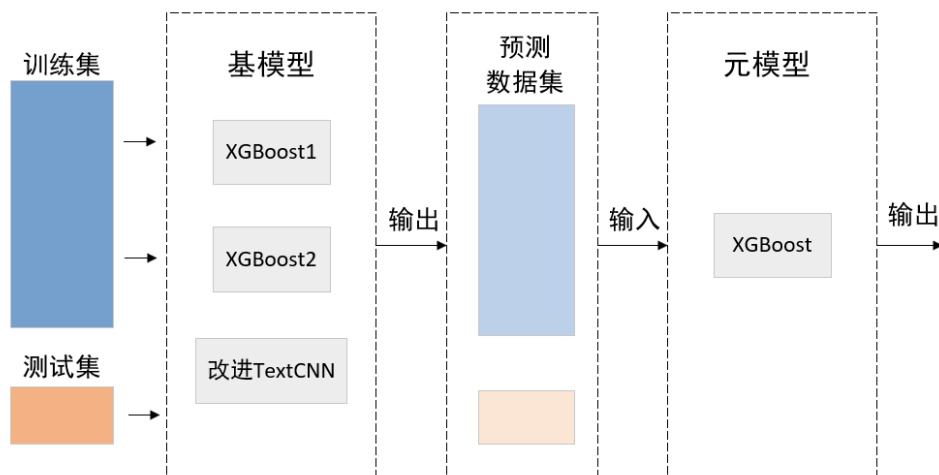


图 4.6 采用 Stacking 集成学习进行模型融合流程图

具体步骤如下：

1. 选取一组基模型，也称为一级模型或底层模型。这些模型既可以是机器学习模型，也可以是深度学习模型。本文选用了 XGBoost 作为机器学习模型，因其在处理结构化的 API 调用序列和提升训练速度方面展现出优势；同时，选择改进 TextCNN 作为深度学习模型，由于其在分析恶意代码文本特征上的出色表现。

2. 对每个基模型使用完整的训练数据集进行独立训练。它们从数据中学习不同的特征表示和模式，并输出预测结果，这些结果作为元特征为下一步的模型训练提供信息。其中 XGBoost 模型分别使用网格搜索与贝叶斯优化技术得到的最佳模型性能参数进行训练，得到两组元特征。

3. 选择一个或多个元模型，也称为二级模型或顶层模型，来整合一级模型的预测。在本文中采用 XGBoost 作为元模型，被训练以学习如何最优化地结合基模型的预测结果，并进行最终的分类。

4. 利用训练好的二级模型对测试集进行预测，并通过各种评估指标如准确率、召回率和 F1 分数来评价模型性能，以此来验证模型在检测恶意代码方面的有效性。

综上所述，本文通过融合不同模型的预测结果，成功实现了一种采用模型融合的恶意代码检测方法。借助 Stacking 集成学习策略，该方法超越了任何单一模型的性能，可以有效提高未知变种恶意代码的检测率，为终端安全防护提供了有力

支撑。

4.4 实验与结果分析

本章首先对实验环境配置、所使用的数据集种类及机器学习中常见的评估指标进行了介绍。随后,通过实验步骤对本文所提出的模型性能进行了验证和分析。本章接着探讨了改进措施对于 TextCNN 模型的准确性的作用,同时考察了 Stacking 集成学习策略对模型融合效果的影响。最后对所提出方法进行了归纳与总结。

4.4.1 实验环境与数据

本文的实验环境配置具体如表 4.3 所示。

表 4.3 实验环境参数

实验参数	实验数值
操作系统	Ubuntu 18
编程语言	Python 3.8.10
开发框架	Keras, TensorFlow, Sklearn
GPU	GeForce RTX 2080 Ti
GPU 数量	5
内存	72GB
硬盘	200GB

本文使用的数据集源自于阿里云安全恶意程序检测挑战赛,该数据集由 Windows 平台下的二进制可执行文件构成,它们在沙箱环境中运行后生成的 API 调用序列。数据集涵盖了 13887 个样本文件,其中包含近九千万条 API 调用记录。在这些样本文件中,4978 个文件被标记为正常,而 8909 个文件被识别为恶意。恶意代码样本统计如表 4.4 所示。

本文将数据集分为训练集和测试集两部分,训练集包含 11109 个样本,用于模型训练;测试集包含 2778 个样本,用于评估模型性能,划分比例约为 4:1。

表 4.4 恶意代码样本统计

序号	类型	数量
0	正常样本	4978
1	勒索病毒	502
2	挖矿程序	1196
3	DDoS 木马	820
4	蠕虫病毒	100
5	感染型病毒	4289
6	后门程序	515
7	木马程序	1487

4.4.2 评价方法及指标

在机器学习领域，评估恶意代码检测模型性能的关键在于使用准确的评估指标。其中，准确率（Accuracy）、精准率（Precision）、召回率（Recall）和 F1 分数（F1 Score）是较为常见的四种评估指标。这些指标基于四个基本概念计算得出：真正例（True Positive, TP）、真负例（True Negative, TN）、假正例（False Positive, FP）和假负例（False Negative, FN）。

首先，准确率是指模型正确识别样本的比例，无论是正例还是负例。它是通过将所有正确预测的结果（TP 和 TN）除以总样本数来计算的。准确率能够直观反映出模型在整体上的性能，即模型对样本分类的总体正确率。计算公式分别如（4-8）所示。

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{TN} + \text{FP}} \quad (\text{式 4-8})$$

其次，精准率专注于模型对正例的预测能力。它是指在所有被模型预测为正例的样本中，实际为正例的样本比例。精准率高意味着模型在预测为正例的样本中具有较高的可靠性。计算公式分别如（4-9）所示。

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (\text{式 4-9})$$

接着，召回率评估的是模型对实际正例的识别能力。它是指在所有实际为正例的样本中，被模型正确预测为正例的样本比例。高召回率表明模型能够较好地捕捉到正例样本，减少漏检的情况。计算公式分别如（4-10）所示。

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (\text{式 4-10})$$

最后，F1 分数（F1 Score）也是一个关键的评估指标。F1 分数是精准率和召回率的调和平均值，用于综合反映模型的精准度和召回能力。特别是在正负样本分布不平衡的情况下，F1 分数能够提供比单一指标更为全面的性能评估。计算公式分别如（4-11）所示。

$$\text{F1} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (\text{式 4-11})$$

4.4.3 结果分析

（1）改进 TextCNN 对检测结果的影响

本实验利用准确率、精准率、召回率和 F1 分数四个指标对模型性能进行评估。在 TextCNN 模型中引入自注意力层与标签平滑技术，通过设计的对比实验，评估了改进 TextCNN 对模型性能的影响。

表 4.5 改进 TextCNN 对检测结果的影响

模型	准确率	精准率	召回率	F1 分数
TextCNN	89.46%	88.88%	89.46%	89.05%
TextCNN+SelfAttention	89.74%	89.28%	89.74%	89.35%
TextCNN+LabelSmoothing	89.91%	89.54%	89.91%	89.52%
本文改进的 TextCNN	90.36%	89.82%	90.36%	89.98%

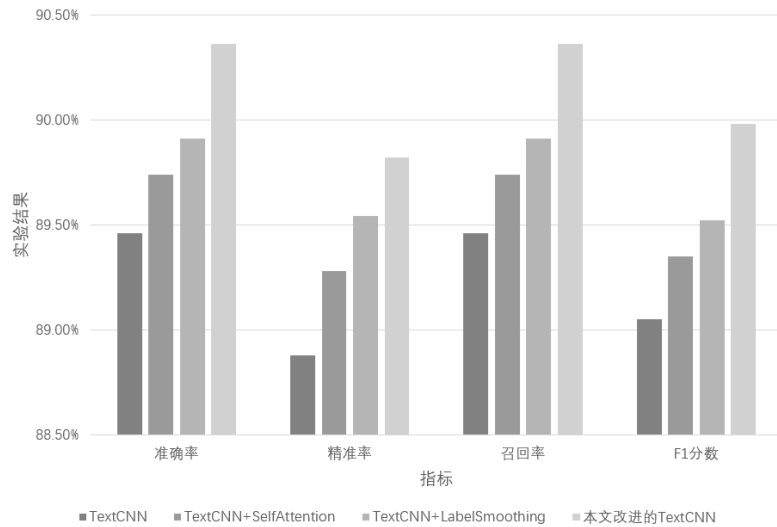


图 4.7 改进 TextCNN 对检测结果的影响

如表 4.5 和图 4.7 是 TextCNN 模型在改进前和改进后的模型性能比较。结果表明,各模型在准确率、精准率、召回率和 F1 分数上的表现揭示了以下发现:TextCNN 已经达到了较高的性能水平,具体体现为 89.46% 的准确率,88.88% 的精准率,89.46% 的召回率以及 89.05% 的 F1 分数。当引入 SelfAttention 机制至 TextCNN 模型中,所有性能指标均获得提升,准确率增加至 89.74%,精准率提高至 89.28%,召回率和 F1 分数分别提升至 89.74% 和 89.35%,表明 SelfAttention 机制通过捕获文本中的长距离依赖关系有效提升了模型性能。进一步地,结合 LabelSmoothing 技术的 TextCNN 模型在所有指标上再次提升,准确率达 89.91%,精准率为 89.54%,召回率和 F1 分数分别为 89.91% 和 89.52%,显示出 LabelSmoothing 技术通过减少模型对硬标签的依赖,增强了模型的泛化能力。最终,经本文改进的 TextCNN 模型在所有评估指标上均达到最高性能,准确率为 90.36%,精准率为 89.82%,召回率和 F1 分数分别为 90.36% 和 89.98%,这一结果证实了改进措施在提升 TextCNN 模型性能方面的有效性,尤其是在准确率和 F1 分数上的提升,为文本分类任务提供了新的改进方向。

(2) 模型融合对检测结果的影响

本实验仍然通过准确率、精准率、召回率和 F1 分数四个指标,对提出的模型与 XGBoost 模型、改进 TextCNN 模型进行了对比分析。

表 4.6 模型融合对检测结果的影响

模型	准确率	精准率	召回率	F1 分数
改进 TextCNN	90.36%	89.82%	90.36%	89.98%
XGBoost1	91.53%	91.26%	91.53%	91.33%
XGBoost2	91.54%	91.32%	91.54%	91.35%
本文提出的模型	92.04%	91.84%	92.04%	91.91%

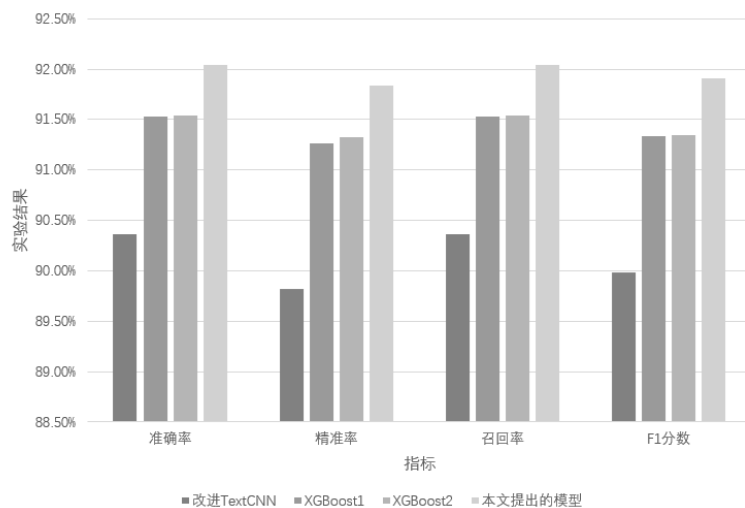


图 4.8 模型融合对检测结果的影响

根据表 4.6 和图 4.8 所示的模型准确率比较, 可以观察到本文提出的模型在准确率、精准率、召回率以及 F1 得分上均优于 XGBoost 和改进 TextCNN 模型。具体而言, XGBoost1 模型展现了 91.53% 的准确率, 91.26% 的精准率, 91.53% 的召回率以及 91.33% 的 F1 分数, 而 XGBoost2 模型在所有指标上略有提升, 达到 91.54% 的准确率, 91.32% 的精准率, 91.54% 的召回率和 91.35% 的 F1 分数。这表明, 尽管两个 XGBoost 模型性能相近, 但是细微的调整可能导致性能的轻微提高。改进 TextCNN 模型的性能稍微低于 XGBoost 模型, 具体体现为 90.36% 的准确率, 89.82% 的精准率, 90.36% 的召回率和 89.98% 的 F1 分数, 这可能是由于 TextCNN 模型在处理特定类型的文本数据时, 未能像 XGBoost 模型那样充分利用其结构优

势。而本文提出的模型在所有评估指标上均实现了最高性能，具体为 92.04% 的准确率，91.84% 的精准率，92.04% 的召回率和 91.91% 的 F1 分数，这一结果表明了该模型在处理给定任务时的出色能力，其改进措施和结构设计显著提升了模型对数据的理解和分类准确性，本文提出的模型在处理复杂的恶意代码检测任务时，展现了较强的性能和泛化能力。

4.5 本章小结

本章详细介绍了一种 Windows 终端安全领域中未知变体恶意代码检测的方法。通过融合 XGBoost 与改进 TextCNN 模型，并采用 Stacking 集成学习策略，本文成功构建了一个有效的恶意代码检测模型。该方法首先利用 XGBoost 和改进 TextCNN 基模型对恶意代码中的 API 调用序列进行训练并生成元特征。随后，通过 XGBoost 元模型对这些元特征进行二次训练，实现了对恶意代码未知变体的有效检测。

本方法的核心优势在于其能够融合不同模型的预测结果，提高检测的准确性和泛化能力。Stacking 集成学习策略的应用，为终端安全领域中的恶意代码检测提供了一种新的解决方案，具有较高的实际应用价值。

第五章 终端恶意代码特征检测系统设计与实现

5.1 需求分析

随着网络攻击技术的持续演进，恶意代码开发者不断采用更为复杂和隐蔽的手段进行攻击，这些变化使得传统的 Windows 终端安全防护策略面临着前所未有的挑战。在这种背景下，恶意代码的检测方法也必须随之发展以适应新的威胁环境。静态检测技术，虽然在某些情况下能够提供高准确率的恶意代码识别，但其固有的局限性使其在面对加壳、混淆等高级逃避技术时显得力不从心。这些技术能够有效地隐藏恶意代码的真实意图，从而绕过基于静态特征的检测机制。相较之下，基于行为分析的检测技术提供了一种更为有效的对策。通过监测和分析恶意代码在执行过程中的行为特征，这种方法能够揭示出恶意代码的真实功能和意图，即使在恶意代码使用了复杂的逃避技术之后也是如此。行为分析技术不仅能够识别已知的恶意代码样本，而且还具有发现采用了免杀技术恶意代码的能力，这得益于其能够捕捉到恶意行为模式而不仅仅是依赖于恶意代码的静态特征。

另外，用户态挂钩绕过与无文件攻击目前在恶意代码上使用的越来越多。攻击通常利用系统内存或合法进程来执行代码，而不必在文件系统中留下明显的痕迹。这使得检测变得更加困难，因为传统的防御工具通常依赖于检查文件的存在和内容。这些攻击方式不仅具有高度的隐蔽性，而且其进化速度迅猛，对抗起来具有一定的挑战。组织需要不断提升其安全意识，采用前沿的安全技术和工具，以及建立完善的安全策略，以更好地防范和对抗无文件攻击所带来的危害。

为了进一步增强恶意代码检测的效率和准确性，引入了机器学习技术，将其作为一种强有力的工具用于分析和识别恶意行为。机器学习算法能够从大量的数据样本中学习恶意代码的特征和行为模式，自动识别出潜在的恶意代码。这不仅包括已知的恶意代码样本，还能够识别出新的、未知的变种恶意代码，提高了检测的覆盖范围和准确率。通过应用机器学习算法，如决策树算法和卷积神经网络，能够更深入地理解恶意代码的复杂行为模式，提取高维度的特征，从而在更复杂的攻击场景下实现有效的恶意代码检测。

本节主要对论文的需求进行了分析,结合本文研究背景和前期的技术研究,确定了基于行为特征恶意代码检测算法以及基于 Stacking 集成学习的恶意代码检测模型。因此本章提出了终端恶意代码特征检测系统的构想,将最新的行为特征检测技术与机器学习检测模型相结合,多重防线应对日益增强的恶意代码威胁。

5.2 系统设计与实现

本节主要设计并实现终端恶意代码特征检测系统,结合了第三章基于行为特征的检测算法和第四章基于 Stacking 集成学习的检测模型。该系统利用了行为特征以及 API 调用序列的特征,从多个维度对恶意代码进行检测。如图 5.1 是终端恶意代码特征检测系统的框架。

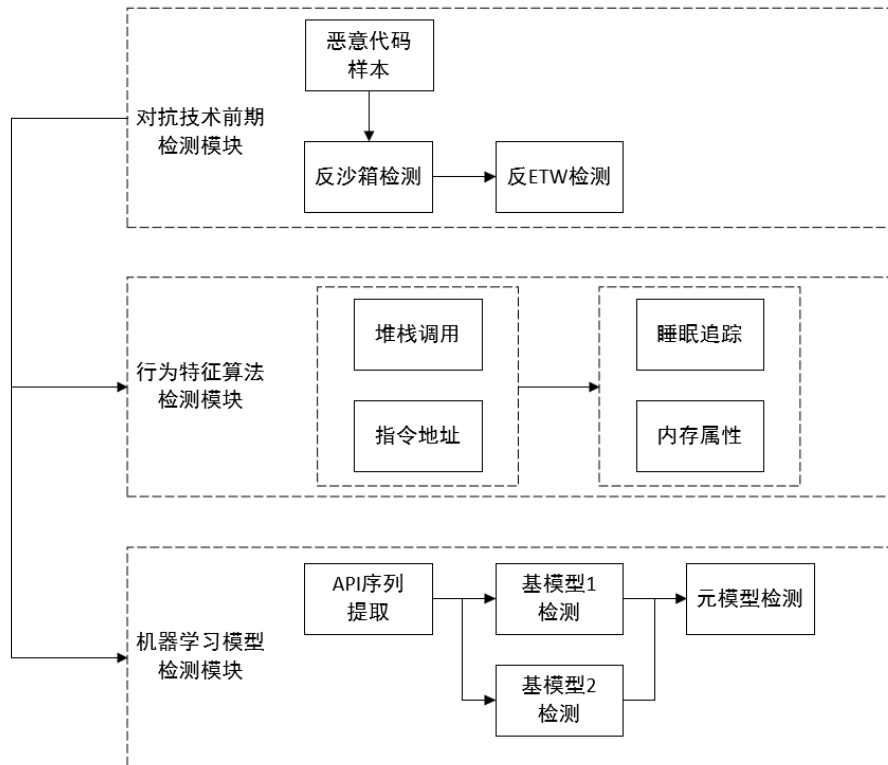


图 5.1 系统框架图

系统由对抗技术前期检测模块、行为特征算法检测模块和机器学习模型检测模块三个主要模块构成。

(1) 在对抗技术前期检测模块,通过监测恶意代码的行为来识别其是否采用了

反 ETW 与反沙箱等对抗技术。通过观察系统的异常行为模式和特定的指标变化,能够及时发现这些对抗技术的存在,并采取相应的反制措施,以确保系统后续检测模块的正常进行。

(2) 在行为特征算法检测模块,从堆栈调用和指令地址角度在恶意代码动态加载时进行识别,并及时采取措施进行拦截与清除;随后,从内存属性和堆栈追踪角度在内存运行阶段对内存中驻留的恶意代码进行追踪与识别,以评估终端内存是否遭到恶意攻击。

(3) 在机器学习模型检测模块中,将恶意代码的 API 调用序列视为文本数据进行处理。该模块运用 Stacking 集成学习策略,首先利用 XGBoost 和改进 TextCNN 模型在第一层进行特征提取和初步训练,从而生成元特征。然后,在第二层使用 XGBoost 元模型对这些元特征进行更深入的学习。通过综合各基模型的检测优势,模块能够有效地应对各种复杂的恶意代码攻击。

5.3 系统测试

5.3.1 测试环境

本文的测试环境具体参数如表 5.1 所示。

表 5.1 实验环境配置

组件设施	型号与参数
CPU	Intel(R) Core(TM) i5-1235U 2.50 GHz
内存	16GB
硬盘	256GB SSD + 1TB HDD
操作系统	Windows 10 64 位

在测试环境中测试的 AV/EDR 分别为 Microsoft Windows Defender、Kaspersky Endpoint Security 以及 Elastic Defend。

首先,对现有免杀技术进行功能性测试,通过在各 AV/EDR 和本章提出的检

测系统上应用多种免杀技术组合，横向比较其对 AV/EDR 的绕过效果。随后，采用近期真实恶意代码样本，对本章设计的检测系统进行性能测试，从而评估其在实际环境中的检测效果。

5.3.2 针对现有免杀技术的功能测试

在本节中，针对当前主流的免杀技术进行了功能测试，旨在评估这些技术对各类 AV/EDR 系统以及本章提出的检测系统的检测效果。使用第 3.4.1 节介绍的开源项目技术，结合多种免杀技术形成的综合免杀方案。将开源命令与控制框架 Havoc 作为测试控制端。（考虑到本章检测主要集中于行为特征，测试采取了远程/本地分离加载的方法，将混淆过的恶意 Shellcode 传输至测试机以规避 AV/EDR 的静态特征检测。）表 5.2 是多种免杀技术组合的样本方案。

表 5.2 组合免杀技术的样本方案

组合免杀技术	
方案一	SysWhispers3+ShellcodeFluctuation+ThreadStackSpoofers
方案二	SysWhispers3+Foliage
方案三	SysWhispers3+Ekko
方案四	HWSyscalls+ShellcodeFluctuation+ThreadStackSpoofers
方案五	HWSyscalls+Foliage
方案六	HWSyscalls+Ekko

经过对样本的检测，表 5.3 是各类 AV/EDR 系统以及本章提出的检测系统的检测结果。

表 5.3 针对现有免杀技术的功能测试

	Microsoft Windows Defender	Kaspersky Endpoint Security	Elastic Defend	本章实现的 检测系统
方案一	检出	检出	检出	检出
方案二	未检出	未检出	检出	检出
方案三	未检出	未检出	检出	检出
方案四	检出	检出	检出	检出
方案五	未检出	未检出	未检出	检出
方案六	未检出	未检出	未检出	检出

5.3.3 恶意代码特征检测系统的性能测试

本节主要对终端恶意代码特征检测系统进行性能测试，目的是评估该检测系统在实际环境中对真实钓鱼木马的检测效果。测试样本主要来源于微步在线的钓鱼木马，选取了近期的 5 个真实样本进行了检测。表 5.4 是从微步在线获取到的钓鱼木马样本信息。

表 5.4 钓鱼木马的样本信息

	样本 MD5	样本提交时间
样本一	2df819466cb82505947855e13e231220	2023/11/07
样本二	62f1ef3c11d08883baa047c06cbdc870	2023/11/17
样本三	40af9c112a5b22bf093198a700b7958a	2023/11/25
样本四	ad334a4b682f59d5eb26e20bbf776af8	2023/12/13
样本五	427e0e4ec5924ab6630904a57cc6b8c2	2024/01/03

经过对样本的检测，表 5.5 是终端恶意代码特征检测系统的检测结果。

表 5.5 终端恶意代码特征检测系统的检查结果

	行为特征算法检测阶段	机器学习模型检测阶段	总体
样本一	检出	检出	检出
样本二	检出	检出	检出
样本三	检出	检出	检出
样本四	检出	检出	检出
样本五	检出	检出	检出

5.4 测试结果分析

本节是针对现有免杀技术的功能测试和恶意代码特征检测系统的性能测试的结果分析，以下是详细的分析：

(1) 针对现有免杀技术的功能测试结果表明，本章实现的检测系统能够在所有情况下成功检出恶意行为，展现出较好的检测能力和适应性。相比之下，如 Microsoft Windows Defender、Kaspersky Endpoint Security 和 Elastic Defend 等其他系统在某些方案中出现了漏检，特别是在较为复杂的方案五和方案六中。这一结果表明本章提出的检测系统面对复杂免杀技术时的有效性，它能够为终端安全提供可靠的保护，成为传统规则检测的有效补充。

(2) 终端恶意代码特征检测系统的性能测试结果表明，所有五个样本均在总体上成功被检出，说明检测系统具备较高的准确性。行为特征算法检测阶段准确识别了所有样本，有效地捕捉了恶意代码的行为特征。机器学习模型检测阶段同样表现出色，成功识别样本一至样本五，显示了其在处理多样化恶意代码上的广泛适应性和显著效果。检出结果不仅证实了所采用方法的有效性，也突显了该系统在多维度分析中的可靠性和全面性。

总体而言，测试结果证明了终端恶意代码特征检测系统在应对复杂恶意代码攻击方面的能力，尤其是其融合对抗技术前期检测、机器学习模型及行为特征算法的多层次检测策略的有效性。

5.5 本章小结

在本章中，主要实现并设计了终端恶意代码特征检测系统，其系统架构包括对抗技术前期检测模块、行为特征算法检测模块和机器学习模型检测模块。这些模块相互结合，构建了一个全面的终端恶意代码检测解决方案。与传统的检测系统相比，该系统能够发现这些新型的、变种的恶意代码，从而提高了恶意代码检测的准确性和及时性，为未来研究和应用恶意代码检测技术提供了参考。

第六章 全文总结与展望

6.1 研究工作总结

随着信息技术的持续进步，终端安全逐渐成为国家重视的领域。恶意代码的种类与数量急剧增加，其传播手段变得日趋复杂多变，迫切需要一种能够迅速且精确地识别恶意代码的方法，以保障网络环境的安全。因此，本文从传统规则检测和机器学习检测出发，结合两者进行综合探讨，对终端安全领域中的恶意代码特征检测进行了研究。

本文的主要工作如下：

(1) 本文调研了恶意代码检测的技术，剖析了恶意代码利用的动态执行和内存加密等免杀技术，同时探讨了机器学习和深度学习模型在恶意代码检测中的应用，以及集成学习策略的效用。通过综合分析近年来的相关研究方法，本文吸取了各种方法的优点，弥补了其中不足，并针对恶意代码检测问题提出了新的视角和解决策略。

(2) 在传统规则检测方面，本文提出了基于行为特征的恶意代码检测算法。通过准确定义恶意行为的特征规则，该系统能够在恶意代码的动态加载阶段准确识别出直接或间接的系统调用，及时阻断恶意活动的执行路径。并还能在内存运行阶段从睡眠追踪和内存属性的角度对恶意代码行为进行检测，为传统规则检测提供了很好的补充。

(3) 在机器学习检测方面，本文提出了基于 Stacking 集成学习的恶意代码检测模型。通过融合 XGBoost 与改进 TextCNN 模型，提高了对未知变体恶意代码的识别准确率。该模型不仅利用了 XGBoost 强大的数据拟合能力，还结合了改进 TextCNN 模型对文本数据的深层次理解能力，通过 Stacking 集成学习策略，实现了对恶意代码的高准确率识别，为未知恶意代码变体的防护提供了有效的技术支撑。

(4) 在多维度检测方面，本文构建的终端恶意代码特征检测系统集成了传统规则检测和机器学习检测的优势，形成了一个全面的恶意代码特征检测系统。该系统首先通过规则检测算法分析恶意代码中的免杀行为特征，然后利用机器学习模

型对恶意代码的未知变种进行分类，全方位提升了对恶意代码的检测能力。多维度的检测策略有效应对了采用免杀技术的未知变种恶意代码，展现出了良好的检测效果。

6.2 研究展望

在研究过程中，针对恶意代码的特征检测取得了一定的成果，但在样本的多样性和模型的可解释性方面仍有待进一步提升。因此，对未来研究方向的展望主要集中在这两个方面：

样本的多样性：未来的研究需要关注于收集和分析更多样化的恶意代码样本。随着恶意代码技术的不断进化，新的变种和类型层出不穷，这就要求我们的样本数据库能够不断更新，以覆盖更广泛的恶意行为模式。增强样本的多样性不仅可以提高模型的泛化能力，也能够帮助研究人员深入理解恶意代码的演化趋势，为未来的防御策略提供支持。

机器学习的可解释性：随着机器学习技术在恶意代码检测中的广泛应用，其模型的可解释性成为了一个重要的研究方向。目前许多高效的机器学习模型，如深度学习网络，其决策过程往往是黑箱的，这在实际应用中可能导致信任度不足。未来的研究可以探索如何提高模型的可解释性，例如，通过开发新的算法来揭示模型的决策逻辑，或使用可视化技术来呈现模型是如何从特征中学习和做出决策的。提高模型的可解释性不仅能够增强对模型判断的信任，也有助于研究人员更好地分析和改进模型，以应对复杂多变的恶意代码攻击。

参考文献

- [1] 瑞星. 2023 年中国网络安全报告 [EB/OL].<http://it.rising.com.cn/d/file/pdf/2023baogao.pdf>, 2024-02-04.
- [2] Hassan W U, Bates A, Marino D. Tactical provenance analysis for endpoint detection and response systems[C]//2020 IEEE Symposium on Security and Privacy (SP). IEEE, 2020: 1172-1189.
- [3] Hai T H, Van Thieu V, Duong T T, et al. A proposed new endpoint detection and response with image-based malware detection system[J]. IEEE Access, 2023, 11: 122859-122875.
- [4] 肖玉强, 郭云飞, 王亚文. 一种基于操作码 N-gram 特征的代码混淆技术识别模型 [J]. 信息工程大学学报, 2023, 24(01): 72-80.
- [5] Sung A H, Xu J, Chavez P, et al. Static analyzer of vicious executables (save)[C]//20th Annual Computer Security Applications Conference. IEEE, 2004: 326-334.
- [6] Karnik A, Goswami S, Guha R. Detecting obfuscated viruses using cosine similarity analysis[C]//First Asia International Conference on Modelling Simulation (AMS'07). IEEE, 2007: 165-170.
- [7] 王晓洁, 王海峰. 基于语义的恶意代码检测算法研究 [J]. 计算机系统应用, 2009, 18(08): 103-106.
- [8] 王蕊, 冯登国, 杨轶等. 基于语义的恶意代码行为特征提取及检测方法 [J]. 软件学报, 2012, 23(02): 378-393.
- [9] Cesare S, Xiang Y, Zhou W. Control flow-based malware variant detection[J]. IEEE Transactions on Dependable and Secure Computing, 2013, 11(4): 307-317.

- [10] Souri A, Hosseini R. A state-of-the-art survey of malware detection approaches using data mining techniques[J]. Human-centric Computing and Information Sciences, 2018, 8(1): 1-22.
- [11] Kim C W. Ntmaldetect: A machine learning approach to malware detection using native api system calls[J]. arXiv preprint arXiv:1802.05412, 2018.
- [12] Vu D H. Privacy-preserving Naive Bayes classification in semi-fully distributed data model[J]. Computers Security, 2022, 115: 102630.
- [13] Fan M, Liu J, Luo X, et al. Android malware familial classification and representative sample selection via frequent subgraph analysis[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(8): 1890-1905.
- [14] Wadkar M, Di Troia F, Stamp M. Detecting malware evolution using support vector machines[J]. Expert Systems with Applications, 2020, 143: 113022.
- [15] 孙博文, 黄炎裔, 温俏琨等. 基于静态多特征融合的恶意软件分类方法 [J]. 网络与信息安全学报, 2017, 3(11): 68-76.
- [16] Chen Z, Zhang X, Kim S. A learning-based static malware detection system with integrated feature[J]. Intell. Autom. Soft Comput, 2021, 27(3): 891-908.
- [17] Louk M H L, Tama B A. Tree-based classifier ensembles for PE malware analysis: A performance revisit[J]. Algorithms, 2022, 15(9): 332.
- [18] Kamboj A, Kumar P, Bairwa A K, et al. Detection of malware in downloaded files using various machine learning models[J]. Egyptian Informatics Journal, 2023, 24(1): 81-94.
- [19] Son T T, Lee C, Le-Minh H, et al. An enhancement for image-based malware classification using machine learning with low dimension normalized input images[J]. Journal of Information Security and Applications, 2022, 69: 103308.
- [20] LeCun Y, Bengio Y, Hinton G. Deep learning[J]. nature, 2015, 521(7553): 436-444.

- [21] Nahmias D, Cohen A, Nissim N, et al. Deep feature transfer learning for trusted and automated malware signature generation in private cloud environments[J]. Neural Networks, 2020, 124: 243-257.
- [22] Tekerek A, Yapici M M. A novel malware classification and augmentation model based on convolutional neural network[J]. Computers Security, 2022, 112: 102515.
- [23] Jindal C, Salls C, Aghakhani H, et al. Neurlux: dynamic malware analysis without feature engineering[C]//Proceedings of the 35th Annual Computer Security Applications Conference. 2019: 444-455.
- [24] Xiaofeng L, Fangshuo J, Xiao Z, et al. ASSCA: API sequence and statistics features combined architecture for malware detection[J]. Computer Networks, 2019, 157: 99-111.
- [25] Sewak M, Sahay S K, Rathore H. An investigation of a deep learning based malware detection system[C]//Proceedings of the 13th International Conference on Availability, Reliability and Security. 2018: 1-5.
- [26] Tobiyama S, Yamaguchi Y, Shimada H, et al. Malware detection with deep neural network using process behavior[C]//2016 IEEE 40th annual computer software and applications conference (COMPSAC). IEEE, 2016, 2: 577-582.
- [27] Catak F O, Yazı A F, Elezaj O, et al. Deep learning based Sequential model for malware analysis using Windows exe API Calls[J]. PeerJ Computer Science, 2020, 6: e285.
- [28] 王天歌. 基于 API 调用序列的 Windows 平台恶意代码检测方法 [D]. 北京交通大学, 2021. DOI:10.26944/d.cnki.gbfju.2021.002779.
- [29] Xu Z, Ren K, Qin S, et al. CDGDroid: Android malware detection based on deep learning using CFG and DFG[C]//Formal Methods and Software Engineering: 20th International Conference on Formal Engineering Methods, ICFEM 2018, Gold Coast, QLD, Australia, November 12-16, 2018, Proceedings 20. Springer International Publishing, 2018: 177-193.

- [30] 张孟哲. 基于深度学习的恶意代码家族分类方法研究 [D]. 东南大学,2022.DOI:10.27014/d.cnki.gdnau.2022.004619.
- [31] Grosse K, Papernot N, Manoharan P, et al. Adversarial perturbations against deep neural networks for malware classification[J]. arXiv preprint arXiv:1606.04435, 2016.
- [32] Gibert D, Mateu C, Planes J. HYDRA: A multimodal deep learning framework for malware classification[J]. Computers Security, 2020, 95: 101873.
- [33] Li W, Zhang C, Zhou J, et al. Malicious Code Detection Method Based on Static Features and Ensemble Learning[C]//Journal of Physics: Conference Series. IOP Publishing, 2021, 2010(1): 012165.
- [34] Nissim N, Lahav O, Cohen A, et al. Volatile memory analysis using the MinHash method for efficient and secured detection of malware in private cloud[J]. Computers Security, 2019, 87: 101590.
- [35] Microsoft. About Event Tracing[EB/OL]. Available: <https://docs.microsoft.com/en-us/windows/win32/etw/about-event-tracing>, 2018-05-31.
- [36] Karantzas G, Patsakis C. An empirical assessment of endpoint detection and response systems against advanced persistent threats attack vectors[J]. Journal of Cybersecurity and Privacy, 2021, 1(3): 387-421.
- [37] Breiman L. Random forests[J]. Machine learning, 2001, 45: 5-32.
- [38] Garcia F C C, Muga II F P. Random forest for malware classification[J]. arXiv preprint arXiv:1609.07770, 2016.
- [39] Chen T, He T, Benesty M, et al. Xgboost: extreme gradient boosting[J]. R package version 0.4-2, 2015, 1(4): 1-4.
- [40] Sagi O, Rokach L. Approximating XGBoost with an interpretable decision tree[J]. Information sciences, 2021, 572: 522-542.

- [41] Okolica J, Peterson G. A compiled memory analysis tool[C]//Advances in Digital Forensics VI: Sixth IFIP WG 11.9 International Conference on Digital Forensics, Hong Kong, China, January 4-6, 2010, Revised Selected Papers 6. Springer Berlin Heidelberg, 2010: 195-204.
- [42] Akhtar M S, Feng T. Detection of malware by deep learning as CNN-LSTM machine learning techniques in real time[J]. Symmetry, 2022, 14(11): 2308.
- [43] 赵静, 李俊, 龙春等. 基于集成 SVM 和 Bagging 的未知恶意流量检测 [J]. 计算机系统应用, 2022, 31(10): 51-59. DOI: 10.15888/j.cnki.csa.008730.
- [44] 易琳, 王欣. 基于 Boosting 集成学习算法的网络入侵检测技术研究 [J]. 现代计算机, 2021, 27(29): 41-45.
- [45] Wolpert D H. Stacked generalization[J]. Neural networks, 1992, 5(2): 241-259.
- [46] 曹波, 李成海, 宋亚飞等. 基于 Stacking 集成学习的网络安全态势预测方法 [J]. 空军工程大学学报, 2022, 23(05): 101-107.
- [47] 张永刚, 吕鹏飞, 张悦等. 基于 Stacking 集成学习的恶意 URL 检测系统设计与实现 [J]. 现代电子技术, 2023, 46(10): 105-109. DOI: 10.16652/j.issn.1004-373x.2023.10.020.
- [48] Bhansali S, Chen W K, De Jong S, et al. Framework for instruction-level tracing and analysis of program executions[C]//Proceedings of the 2nd international conference on Virtual execution environments. 2006: 154-163.
- [49] Zhang Y, Wallace B. A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification[J]. arXiv preprint arXiv:1510.03820, 2015.
- [50] Pereyra G, Tucker G, Chorowski J, et al. Regularizing neural networks by penalizing confident output distributions[J]. arXiv preprint arXiv:1701.06548, 2017.